

FULL STACK APPLICATION DEVELOPMENT
(COURSE CODE:10211CS224)

NAME:M.INDRAJITH

VTU:24668

REG_NO:23UECS0740

SLOT_NO:E3-B9

FACULTY_NAME: Dr.HAFIZUR RAHMAN

TASK 1: Student Registration & Data Storage System

AIM:

To design and develop a Student Registration System using HTML, CSS, PHP, and MySQL to collect, store, and retrieve student details efficiently.

ALGORITHM:

Step 1: Start

Step 2: Open the registration form (index.html)

Step 3: Display input fields (Name, Email, DOB, Department, Phone)

Step 4: User enters the required details

Step 5: Validate input fields

- Ensure fields are not empty
- Validate email format
- Validate phone number

Step 6: Submit the form using POST method

Step 7: Server (PHP) receives the form data

Step 8: Establish connection with MySQL database

- If connection fails → display error
- Else → continue

Step 9: Insert student data into database table

Step 10: Display success message

Step 11: Retrieve stored student data using SELECT query

Step 12: Display data in tabular format

Step 13: Stop

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>Student Registration</title>
```

```
    <link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
<div class="container">

  <h2>Student Registration</h2>

  <form action="insert.php" method="post">

    <label>Name</label>

    <input type="text" name="name" required>

    <label>Email</label>

    <input type="email" name="email" required>

    <label>Date of Birth</label>

    <input type="date" name="dob" required>

    <label>Department</label>

    <select name="department" required>

      <option value="">Select</option>

      <option>Computer Science</option>

      <option>Information Technology</option>

      <option>Electronics</option>

      <option>Mechanical</option>

    </select>

    <label>Phone</label>

    <input type="tel" name="phone" required>

    <button type="submit">Register</button>

  </form>

</div>

</body>

</html>
```

STYLE.CSS:

```
body {

  font-family: Arial, sans-serif;

  background: #f4f6f8;
```

```
}
```

```
.container {
```

```
    width: 400px;
```

```
    margin: 60px auto;
```

```
    background: white;
```

```
    padding: 25px;
```

```
    border-radius: 8px;
```

```
    box-shadow: 0 0 10px rgba(0,0,0,0.1);
```

```
}
```

```
h2 {
```

```
    text-align: center;
```

```
    margin-bottom: 20px;
```

```
}
```

```
label {
```

```
    display: block;
```

```
    margin-top: 10px;
```

```
    font-weight: bold;
```

```
}
```

```
input, select {
```

```
    width: 100%;
```

```
    padding: 8px;
```

```
    margin-top: 5px;
```

```
}
```

```
button {
```

```
    width: 100%;
```

```
    margin-top: 20px;
```

```
    padding: 10px;
```

```
    background: #007bff;
```

```
    color: white;

    border: none;

    cursor: pointer;
}

button:hover {

    background: #0056b3;

}
```

INSERT.PHP:

```
<?php

$conn = new mysqli("localhost", "root", "", "student_db");

if ($conn->connect_error) {

    die("Connection Failed");

}

$name = $_POST['name'];

$email = $_POST['email'];

$dob = $_POST['dob'];

$department = $_POST['department'];

$phone = $_POST['phone'];

$sql = "INSERT INTO students (name, email, dob, department, phone)

        VALUES ('$name', '$email', '$dob', '$department', '$phone')";

if ($conn->query($sql)) {

    echo "Student Registered Successfully";

} else {

    echo "Error";

}

$conn->close();

?>
```

VIEW.PHP:

```
<?php

$conn = new mysqli("localhost", "root", "", "student_db");

$result = $conn->query("SELECT * FROM students");

?>

<!DOCTYPE html>

<html>

<head>

    <title>Student List</title>

</head>

<body>

<h2>Registered Students</h2>

<table border="1" cellpadding="8">

<tr>

    <th>ID</th>

    <th>Name</th>

    <th>Email</th>

    <th>DOB</th>

    <th>Department</th>

    <th>Phone</th>

</tr>

<?php

while ($row = $result->fetch_assoc()) {

    echo "<tr>

        <td>{$row['id']}</td>

        <td>{$row['name']}</td>

        <td>{$row['email']}</td>

        <td>{$row['dob']}</td>

        <td>{$row['department']}</td>
```

```
        <td>{$row['phone']}</td>

    </tr>";

}

?>

</table>

</body>

</html>
```

DATABASE (MySQL):

```
CREATE DATABASE student_db;

USE student_db;

CREATE TABLE students (

    id INT AUTO_INCREMENT PRIMARY KEY,

    name VARCHAR(100),

    email VARCHAR(100),

    dob DATE,

    department VARCHAR(50),

    phone VARCHAR(15)

);
```

OUTPUT:

Student Registration

Name

Arun Kumar

Email

arun.kumar@example.com

Date of Birth

2002-06-15

Department

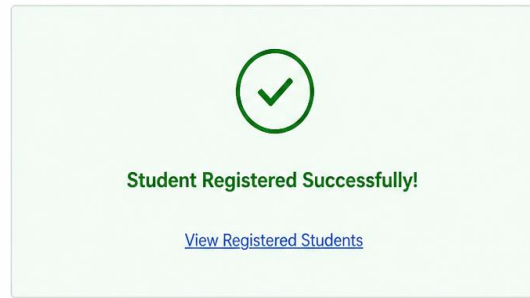
Computer Science

Phone

9876543210

Register

(a) Student Registration Form



(b) Success Message

Registered Students					
ID	Name	Email	DOB	Department	Phone
1	Arun Kumar	arun.kumar@example.com	2002-06-15	Computer Science	9876543210
2	Priya Sharma	priya.sharma@example.com	2001-09-22	Information Technology	9123456780
3	Ravi Patel	ravi.patel@example.com	2002-01-10	Electronics	9988776655
4	Sneha Reddy	sneha.reddy@example.com	2001-12-05	Mechanical	9001122334

(c) Retrieved Student Records

RESULT:

The student registration system was successfully developed to store and retrieve student data using a web interface and MySQL database.

TASK 2: Data Retrieval & Sorting Dashboard

AIM:

To develop a web-based dashboard that retrieves, displays, sorts, and filters student data from a MySQL database and shows department-wise student count.

ALGORITHM

Step 1: Start

Step 2: Connect to MySQL database

Step 3: Retrieve sorting parameter (name/date) from URL

Step 4: Retrieve department filter from user input

Step 5: Construct SQL query

- If filter applied → use WHERE clause
- Else → fetch all records

Step 6: Apply ORDER BY clause for sorting

Step 7: Execute query and fetch results

Step 8: Display student data in table format

Step 9: Execute GROUP BY query

- Count students per department

Step 10: Display count in separate table

Step 11: Stop

PROGRAM:

INDEX.HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Student Registration</title>
```

```
  <link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
  <h2>Student Registration</h2>
```

```
  <form action="insert.php" method="post">
```

```
    <input type="text" name="name" placeholder="Name" required>
```

```
    <input type="email" name="email" placeholder="Email" required>
```

```
    <input type="date" name="dob" required>
```

```
    <select name="department" required>
```

```

        <option value="">Select Department</option>

        <option>CSE</option>

        <option>ECE</option>

        <option>MECH</option>

    </select>

    <input type="text" name="phone" placeholder="Phone" required>

    <button type="submit">Register</button>

</form>

</div>

</body>

</html>

```

INSERT.PHP:

```

<?php

$conn = new mysqli("localhost", "root", "", "student_db");

$name = $_POST['name'];

$email = $_POST['email'];

$dob = $_POST['dob'];

$department = $_POST['department'];

$phone = $_POST['phone'];

$sql = "INSERT INTO students (name, email, dob, department, phone)

        VALUES ('$name', '$email', '$dob', '$department', '$phone')";

$conn->query($sql);

?>

```

DASHBOARD.PHP:

```

<?php

$conn = new mysqli("localhost", "root", "", "college");

// Sorting

```

```

$sort = "name";

if(isset($_GET['sort'])) {

    $sort = $_GET['sort'];

}

// Filtering

$department = "";

if(isset($_GET['department'])) {

    $department = $_GET['department'];

}

// Query

$sql = "SELECT * FROM students";

if($department != "") {

    $sql .= " WHERE department='$department'";

}

$sql .= " ORDER BY $sort";

$result = $conn->query($sql);

?>

<!DOCTYPE html>

<html>

<body>

<h2 align="center">Student Dashboard</h2>

<!-- Sorting -->

<div align="center">

    <a href="?sort=name">Sort by Name</a> |

    <a href="?sort=admission_date">Sort by Date</a>

</div>

<!-- Filter -->

```

```
<form method="GET" align="center">

    <select name="department">

        <option value="">All Departments</option>

        <option value="CSE">CSE</option>

        <option value="ECE">ECE</option>

        <option value="MECH">MECH</option>

    </select>

    <input type="submit" value="Filter">

</form>
```

```
<!-- Table -->
```

```
<table border="1" align="center">
```

```
<tr>
```

```
    <th>ID</th>
```

```
    <th>Name</th>
```

```
    <th>Department</th>
```

```
    <th>Admission Date</th>
```

```
</tr>
```

```
<?php
```

```
while($row = $result->fetch_assoc()) {
```

```
    echo "<tr>
```

```
        <td>{$row['id']}</td>
```

```
        <td>{$row['name']}</td>
```

```
        <td>{$row['department']}</td>
```

```
        <td>{$row['admission_date']}</td>
```

```
    </tr>";
```

```
}
```

```
?>
```

```

</table>

<?php

// Count students per department

$count_sql = "SELECT department, COUNT(*) as total FROM students GROUP BY
department";

$count_result = $conn->query($count_sql);

echo "<h3 align='center'>Student Count Per Department</h3>";

echo "<table border='1' align='center'>";

echo "<tr><th>Department</th><th>Total Students</th></tr>";

while($row = $count_result->fetch_assoc()) {

    echo "<tr>

        <td>{$row['department']}</td>

        <td>{$row['total']}</td>

    </tr>";

}

echo "</table>";

?>

</body>

</html>

```

DATABASE:

```

CREATE DATABASE college;

USE college;

CREATE TABLE students (

    id INT AUTO_INCREMENT PRIMARY KEY,

    name VARCHAR(100),

    department VARCHAR(50),

    admission_date DATE

```

```
);
```

VIEW.PHP:

```
<?php
```

```
$conn = new mysqli("localhost", "root", "", "college");
```

```
// Sorting
```

```
$sort = "name";
```

```
if(isset($_GET['sort'])) {
```

```
    $sort = $_GET['sort'];
```

```
}
```

```
// Filtering
```

```
$department = "";
```

```
if(isset($_GET['department'])) {
```

```
    $department = $_GET['department'];
```

```
}
```

```
// Query
```

```
$sql = "SELECT * FROM students";
```

```
if($department != "") {
```

```
    $sql .= " WHERE department='$department'";
```

```
}
```

```
$sql .= " ORDER BY $sort";
```

```
$result = $conn->query($sql);
```

```
?>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Student Dashboard</title>
```

```
    <link rel="stylesheet" href="style.css">
```

```
</head>

<body>

<h2 class="title">Student Dashboard</h2>

<!-- Sorting -->

<div class="controls">

    <a href="?sort=name">Sort by Name</a>

    <a href="?sort=admission_date">Sort by Date</a>

</div>

<!-- Filter -->

<form method="GET" class="controls">

    <select name="department">

        <option value="">All Departments</option>

        <option value="CSE">CSE</option>

        <option value="ECE">ECE</option>

        <option value="MECH">MECH</option>

    </select>

    <button type="submit">Filter</button>

</form>

<!-- Student Table -->

<table class="table">

<tr>

    <th>ID</th>

    <th>Name</th>

    <th>Department</th>

    <th>Admission Date</th>

</tr>

<?php
```

```

while($row = $result->fetch_assoc()) {

    echo "<tr>

        <td>{$row['id']}</td>

        <td>{$row['name']}</td>

        <td>{$row['department']}</td>

        <td>{$row['admission_date']}</td>

    </tr>";

}

?>

</table>

<?php

// Count per department

$count_sql = "SELECT department, COUNT(*) as total FROM students GROUP BY
department";

$count_result = $conn->query($count_sql);

?>

<h3 class="title">Student Count Per Department</h3>

<table class="table">

<tr>

    <th>Department</th>

    <th>Total Students</th>

</tr>

<?php

while($row = $count_result->fetch_assoc()) {

    echo "<tr>

        <td>{$row['department']}</td>

```



```
        <td>{$row['total']}</td>

    </tr>";

}

?>

</table>

</body>

</html>
```

STYLE.CSS:

```
body {

    font-family: Arial, sans-serif;

    background: #f4f6f8;

    text-align: center;

}

/* Title */

.title {

    margin-top: 20px;

    color: #333;

}

/* Controls (sort + filter) */

.controls {

    margin: 15px;

}

.controls a {

    margin: 10px;

    text-decoration: none;

    color: #007bff;

    font-weight: bold;
```

```
}

.controls a:hover {

    text-decoration: underline;

}

/* Table */

.table {

    width: 70%;

    margin: 20px auto;

    border-collapse: collapse;

    background: white;

    box-shadow: 0 0 10px rgba(0,0,0,0.1);

}

.table th {

    background: #007bff;

    color: white;

    padding: 10px;

}

.table td {

    padding: 10px;

    border: 1px solid #ddd;

}

.table tr:nth-child(even) {

    background: #f9f9f9;

}

}
```

OUTPUT:

Student Registration

Name

Email

Date of Birth

Department

Phone

Student Dashboard

Sort By: Filter by Department:

Student Records

ID	Name	Department	Admission Date
1	Anun Kumar	CSE	2024-06-15
2	Priya Sharma	ECE	2024-06-18
3	Ravi Patel	MECH	2024-06-20
4	Sneha Reddy	CSE	2024-06-22
5	Karan Singh	ECE	2024-06-25
6	Anjali Verma	MECH	2024-06-27
7	Vikram Naik	CSE	2024-06-30

Student Count Per Department

Department	Total Students
CSE	3
ECE	2
MECH	2

RESULT:

The system successfully retrieves, sorts, filters, and displays student data along with department-wise count using a web-based dashboard.

TASK 3: Login System with Validation

AIM

To develop a secure login system that validates user credentials using HTML, CSS, JavaScript, PHP, and MySQL, and allows access only to authorized users.

ALGORITHM

Step 1: Start

Step 2: Display login form (username & password)

Step 3: User enters credentials

Step 4: Perform client-side validation using JavaScript

- Check if fields are empty

Step 5: Submit form data to server

Step 6: Server receives data using PHP

Step 7: Establish connection with MySQL database

Step 8: Execute SQL query to check credentials

Step 9: If credentials match

- Redirect to dashboard page

Step 10: Else

- Display error message

Step 11: Stop

PROGRAM:

LOGIN.HTML:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Login Page</title>
```

```
    <link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
<div class="login-box">
```

```
    <h2>Login</h2>
```

```
    <form id="loginForm" action="login.php" method="POST">
```

```
        <input type="text" id="username" name="username" placeholder="Enter Username">
```

```
        <span id="userError" class="error"></span>
```

```
        <input type="password" id="password" name="password" placeholder="Enter Password">
```

```
        <span id="passError" class="error"></span>
```

```
        <button type="submit">Login</button>
```

```
        <p id="loginError" class="error"></p>
```

```
    </form>
```

```
</div>
```

```
<script>

document.getElementById("loginForm").addEventListener("submit", function(e) {

    let valid = true;

    let username = document.getElementById("username").value;

    let password = document.getElementById("password").value;

    document.getElementById("userError").innerText = "";

    document.getElementById("passError").innerText = "";

    if (username === "") {

        document.getElementById("userError").innerText = "Username is required";

        valid = false;

    }

    if (password === "") {

        document.getElementById("passError").innerText = "Password is required";

        valid = false;

    }

    if (!valid) {

        e.preventDefault();

    }

});

</script>

</body>

</html>
```

STYLE.CSS:

```
body {

    font-family: Arial;

    background: linear-gradient(to right, #4facfe, #00f2fe);

    display: flex;

    justify-content: center;
```

```
    align-items: center;

    height: 100vh;
}

.login-box {

    background: white;

    padding: 30px;

    width: 300px;

    border-radius: 10px;

    text-align: center;
}

input {

    width: 100%;

    padding: 10px;

    margin: 10px 0;
}

button {

    width: 100%;

    padding: 10px;

    background: #4facfe;

    border: none;

    color: white;

    font-weight: bold;

    cursor: pointer;
}

.error {

    color: red;

    font-size: 13px;
}
```

DB.PHP:

```
<?php

$conn = new mysqli("localhost", "root", "", "login_system");

if ($conn->connect_error) {

    die("Connection Failed: " . $conn->connect_error);

}

?>
```

LOGIN.PHP:

```
<?php

include "db.php";

$username = $_POST['username'];

$password = $_POST['password'];

$sql = "SELECT * FROM users WHERE username='$username' AND password='$password'";

$result = $conn->query($sql);

if ($result->num_rows > 0) {

    header("Location: dashboard.php");

} else {

    echo "<script>

        alert('Invalid Username or Password');

        window.location.href='login.html';

    </script>";

}

?>
```

DASHBOARD.PHP:

```
<?php

session_start();

?>

<!DOCTYPE html>
```

```
<html>

<head>

    <title>Dashboard</title>

</head>

<body>

    <h2>Login Successful 🎉</h2>

    <a href="login.html">Logout</a>

</body>

</html>
```

DATABASE (MySQL):

```
CREATE DATABASE login_system;

USE login_system;

CREATE TABLE users (

    id INT AUTO_INCREMENT PRIMARY KEY,

    username VARCHAR(50),

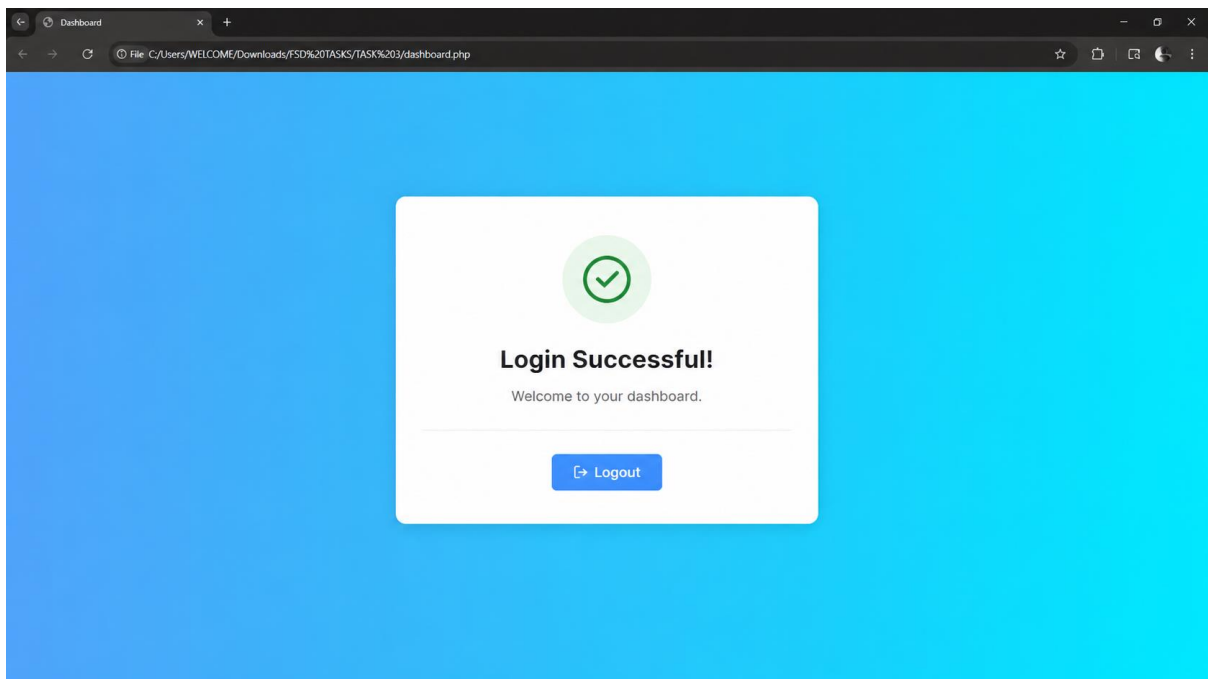
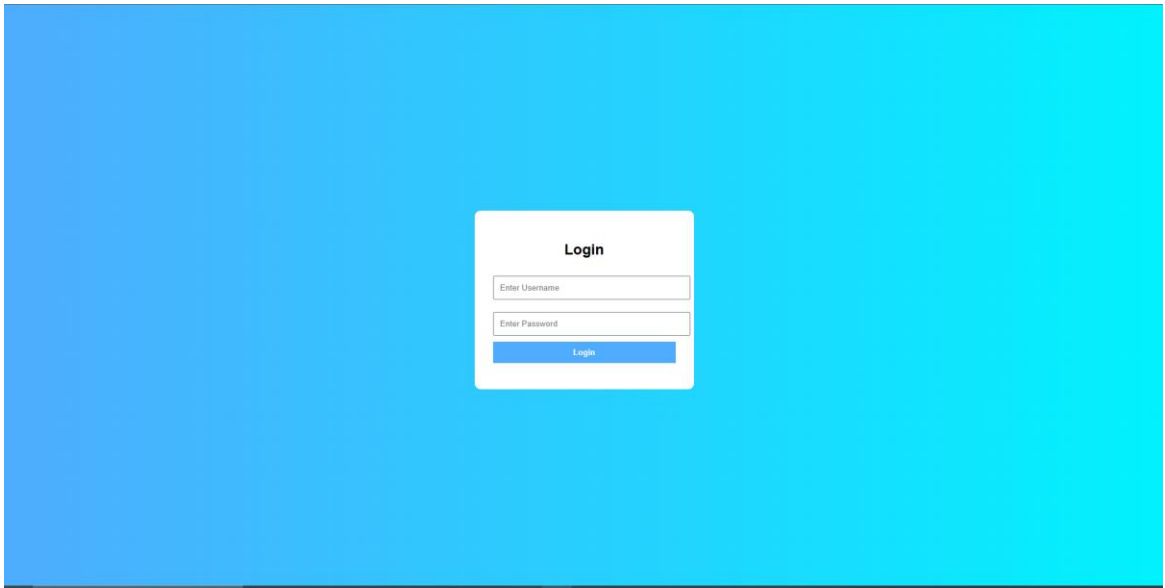
    password VARCHAR(50)

);

INSERT INTO users (username, password)

VALUES ('admin', '1234');
```

OUTPUT:



RESULT:

The login system was successfully developed to validate user credentials and provide secure access using PHP and MySQL.

TASK 4: Order Management using Joins**AIM:**

To design and implement an order management system using SQL joins and subqueries to retrieve customer, product, and order details and analyze data such as highest order value and most active customer.

ALGORITHM:

Step 1: Start

Step 2: Create database and tables

- Customers
- Products
- Orders

Step 3: Insert sample data into tables

Step 4: Establish connection with MySQL database

Step 5: Execute JOIN query

- Combine Customers, Orders, and Products tables
- Display order details

Step 6: Calculate total amount

- Use expression: price × quantity

Step 7: Display order history in table format

Step 8: Find highest order value

- Use MAX() function with subquery

Step 9: Find most active customer

- Use GROUP BY and COUNT()

Step 10: Display results

Step 11: Stop

PROGRAM:

DB.PHP:

```
<?php

$conn = new mysqli("localhost", "root", "", "order_management");

if ($conn->connect_error) {

    die("Connection Failed: " . $conn->connect_error);

}

?>
```

INDEX.PHP:

```
<?php

include "db.php";

?>

<!DOCTYPE html>

<html>

<head>
```

```
<title>Order Management</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<h2>Customer Order History</h2>

<table>

  <tr>

    <th>Customer</th>

    <th>Product</th>

    <th>Price</th>

    <th>Quantity</th>

    <th>Total</th>

    <th>Date</th>

  </tr>

  <?php

  $sql = "SELECT c.name, p.product_name, p.price, o.quantity,

    (p.price * o.quantity) AS total, o.order_date

    FROM orders o

    JOIN customers c ON o.customer_id = c.customer_id

    JOIN products p ON o.product_id = p.product_id

    ORDER BY o.order_date DESC";

  $result = $conn->query($sql);

  while($row = $result->fetch_assoc()) {

    echo "<tr>

      <td>{$row['name']}</td>

      <td>{$row['product_name']}</td>

      <td>{$row['price']}</td>

      <td>{$row['quantity']}</td>

      <td>{$row['total']}</td>

      <td>{$row['order_date']}</td>
```

```

        </tr>";
    }
    ?>
</table>

<hr>

<h3>Highest Value Order</h3>

<?php
$highest = "SELECT MAX(p.price * o.quantity) AS max_order
            FROM orders o
            JOIN products p ON o.product_id = p.product_id";

$res = $conn->query($highest);
$data = $res->fetch_assoc();
echo "<p>₹ " . $data['max_order'] . "</p>";
?>

<hr>

<h3>Most Active Customer</h3>

<?php
$active = "SELECT name FROM customers
            WHERE customer_id = (
                SELECT customer_id
                FROM orders
                GROUP BY customer_id
                ORDER BY COUNT(order_id) DESC
                LIMIT 1
            )";

$res2 = $conn->query($active);
$data2 = $res2->fetch_assoc();
echo "<p>" . $data2['name'] . "</p>";
?>

</body>

```

</html>

STYLE.CSS:

body {

font-family: Arial;

background: #f4f6f9;

padding: 30px;

}

h2 {

text-align: center;

}

table {

width: 100%;

border-collapse: collapse;

margin-top: 20px;

}

th {

background: #4CAF50;

color: white;

padding: 10px;

}

td {

padding: 10px;

text-align: center;

}

tr:nth-child(even) {

background: #f2f2f2;

}

hr {

margin: 40px 0;

}

DATABASE (MySQL):

```
CREATE DATABASE order_management;
```

```
USE order_management;
```

```
CREATE TABLE customers (
```

```
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    name VARCHAR(100)
```

```
);
```

```
CREATE TABLE products (
```

```
    product_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    product_name VARCHAR(100),
```

```
    price INT
```

```
);
```

```
CREATE TABLE orders (
```

```
    order_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    customer_id INT,
```

```
    product_id INT,
```

```
    quantity INT,
```

```
    order_date DATE
```

```
);
```

OUTPUT:

Order Management System

localhost/order_management/login.html



Order Management System

Please login to continue

Username

 Enter your username

Password

 Enter your password 

☐ Remember Me

[Forgot Password?](#)

Login

© 2025 Order Management System. All Rights Reserved.

Order Management

localhost/order_management/index.php

Customer Order History

Customer	Product	Price (₹)	Quantity	Total (₹)	Date
Rahul Sharma	Laptop	50000	1	50000	2024-05-20
Priya Verma	Smartphone	15000	2	30000	2024-05-19
Amit Singh	Headphones	2000	3	6000	2024-05-18
Neha Gupta	Smartwatch	5000	1	5000	2024-05-17
Rahul Sharma	Keyboard	1200	2	2400	2024-05-16
Priya Verma	Mouse	800	2	1600	2024-05-15
Amit Singh	Monitor	12000	1	12000	2024-05-14

Highest Value Order

₹ 50000

Most Active Customer

Rahul Sharma

RESULT

The system successfully retrieves and analyzes order data using SQL joins and subqueries, displaying meaningful insights such as highest order value and most active customer.

TASK 5: Transaction-Based Payment Simulation

AIM:

To simulate an online payment system using SQL transactions to transfer money between accounts, ensuring data consistency using COMMIT and ROLLBACK operations.

ALGORITHM

Step 1: Start

Step 2: Create database and accounts table

Step 3: Insert initial account balances

Step 4: Begin transaction

Step 5: Deduct amount from sender account

Step 6: Add amount to receiver account

Step 7: Check if transaction is valid

- If sufficient balance → COMMIT
- Else → ROLLBACK

Step 8: Display updated balances

Step 9: Stop

PROGRAM:

DATABASE:

```
CREATE DATABASE bank_system;
```

```
USE bank_system;
```

```
CREATE TABLE accounts (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    name VARCHAR(50),
```

```
    balance INT
```

```
);
```

```
-- Insert sample data
```

```
INSERT INTO accounts (name, balance) VALUES
```

```
('Alice', 5000),
```

```
('Bob', 3000);
```

TRANSACTION QUERY:

```
START TRANSACTION;
```

```
-- Deduct from Alice
```

```
UPDATE accounts SET balance = balance - 1000 WHERE name = 'Alice';
```

```
-- Add to Bob
```

```
UPDATE accounts SET balance = balance + 1000 WHERE name = 'Bob';
```

```
-- If everything is correct
```

```
COMMIT;
```

```
-- If error occurs
```

```
-- ROLLBACK;
```

OUTPUT:

The screenshot displays a web application interface for a payment simulation. At the top, a green banner indicates a "Transaction Successful!" with the message "₹1000 has been transferred from Alice to Bob." Below this, the "Transfer Details" section shows the transfer from Alice's account to Bob's account for an amount of ₹1000, accompanied by icons of bank buildings. The interface then compares "Account Balances (Before Transaction)" and "Account Balances (After Transaction)". The "Before" state shows Alice with a balance of 5000 and Bob with 3000. The "After" state shows Alice with a balance of 4000 and Bob with 4000. A "Transaction Log" table at the bottom records the successful transfer. A "Refresh Balances" button is located at the bottom right.

Transaction-Based Payment Simulation
Simulate money transfer between two accounts using SQL Transactions

Transaction Successful!
₹1000 has been transferred from Alice to Bob.

Transfer Details

From Account : Alice
To Account : Bob
Amount : ₹1000

Account Balances (Before Transaction)

ID	Account Holder	Balance (₹)
1	Alice	5000
2	Bob	3000

Account Balances (After Transaction)

ID	Account Holder	Balance (₹)
1	Alice	4000
2	Bob	4000

Transaction Log

Time	Action	Status	Details
2025-05-25 23:20:15	Transfer ₹1000 from Alice to Bob	Success	Transaction committed successfully.

[Refresh Balances](#)

RESULT:

The transaction-based payment system was successfully implemented using SQL transactions, ensuring data integrity with COMMIT and ROLLBACK operations.

TASK 6: Automated Logging using Triggers & Views

AIM:

To implement automated logging of database operations using SQL triggers and to generate reports using views.

ALGORITHM:

Step 1: Start

Step 2: Create database and tables

- Main table (students or transactions)
- Log table (audit_log)

Step 3: Insert initial data

Step 4: Create TRIGGER

- Trigger activates on INSERT or UPDATE
- Automatically stores changes in log table

Step 5: Perform INSERT / UPDATE operations

Step 6: Verify logs are recorded automatically

Step 7: Create VIEW

- Combine or summarize data
- Generate report (daily activity)

Step 8: Retrieve data from VIEW

Step 9: Stop

PROGRAM:

DATABASE + TABLES:

```
CREATE DATABASE audit_system;
```

```
USE audit_system;
```

```
-- Main table
```

```
CREATE TABLE students (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    name VARCHAR(100),
```

```
    department VARCHAR(50)
```

```
);
```

```
-- Log table
```

```
CREATE TABLE audit_log (
```

```
    log_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    action VARCHAR(20),
```

```
    student_id INT,
```

```
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
```

);

TRIGGER (INSERT LOGGING):

DELIMITER \$\$

CREATE TRIGGER after_student_insert

AFTER INSERT ON students

FOR EACH ROW

BEGIN

INSERT INTO audit_log(action, student_id)

VALUES ('INSERT', NEW.id);

END \$\$

DELIMITER ;

TRIGGER (UPDATE LOGGING):

DELIMITER \$\$

CREATE TRIGGER after_student_update

AFTER UPDATE ON students

FOR EACH ROW

BEGIN

INSERT INTO audit_log(action, student_id)

VALUES ('UPDATE', NEW.id);

END \$\$

DELIMITER ;

INSERT / UPDATE TEST:

-- Insert data

INSERT INTO students (name, department)

VALUES ('Arun', 'CSE');

-- Update data

UPDATE students SET department='ECE' WHERE id=1;

VIEW (DAILY ACTIVITY REPORT):

CREATE VIEW daily_activity AS

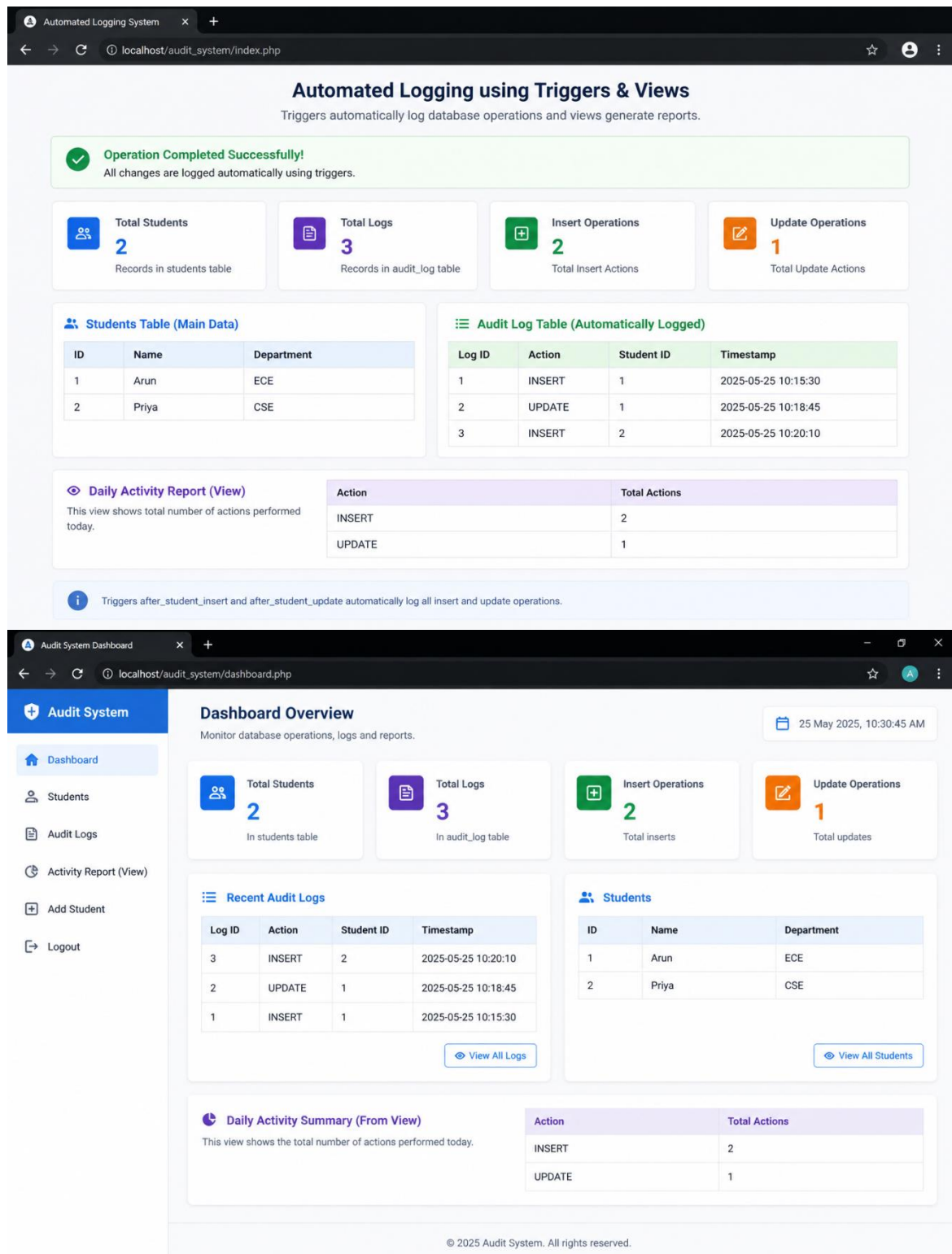
SELECT action, COUNT(*) AS total_actions

FROM audit_log

GROUP BY action;

SELECT * FROM daily_activity;

OUTPUT:



RESULT

The automated logging system was successfully implemented using SQL triggers and views, enabling real-time tracking of database changes and generating summarized reports.

TASK 7: Interactive Web Form with Events & Functions

AIM:

To develop an interactive feedback form using HTML, CSS, and JavaScript that validates user input, responds to events like keypress, hover, and double-click, and improves user experience.

ALGORITHM:

Step 1: Start

Step 2: Create HTML form with input fields

- Name
- Email
- Feedback

Step 3: Apply CSS styling

- Design form layout
- Add hover effects

Step 4: Implement JavaScript functions

- Validate name on keypress
- Validate email format

Step 5: Display error messages dynamically

Step 6: Detect mouse hover event

- Highlight input fields

Step 7: Detect double-click event on submit button

Step 8: If inputs are valid

- Show success message

Step 9: Else

- Show error alert

Step 10: Stop

PROGRAM:

jith.html:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>

  <title>Interactive Feedback Form</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

  <h2>Customer Feedback Form</h2>

  <form id="feedbackForm">

    <input type="text" id="name" placeholder="Enter Name" onkeyup="validateName()">

    <div id="nameError" class="error"></div>

    <input type="email" id="email" placeholder="Enter Email" onkeyup="validateEmail()">

    <div id="emailError" class="error"></div>

    <textarea id="feedback" placeholder="Enter Feedback"></textarea>

    <button type="button" onclick="submitForm()">Submit (Double Click)</button>

  </form>

</div>

<script src="script.js"></script>

</body>

</html>
```

JAVASCRIPT (script.js):

```
// Name Validation

function validateName() {

  let name = document.getElementById("name").value;

  let error = document.getElementById("nameError");

  if (name.length < 3) {

    error.innerHTML = "Name must be at least 3 characters";

  } else {

    error.innerHTML = "";
```



```

    }
}

// Email Validation
function validateEmail() {
    let email = document.getElementById("email").value;
    let error = document.getElementById("emailError");
    let pattern = /^[^ ]+@^[^ ]+\.[a-z]{2,3}$/;
    if (!email.match(pattern)) {
        error.innerHTML = "Enter valid email";
    } else {
        error.innerHTML = "";
    }
}

// Double Click Submit
function submitForm() {
    let nameError = document.getElementById("nameError").innerHTML;
    let emailError = document.getElementById("emailError").innerHTML;
    if (nameError === "" && emailError === "") {
        alert("Thank you! Your feedback has been submitted.");
    } else {
        alert("Please correct errors before submitting.");
    }
}

style.css:
body {
    font-family: Arial;
    background-color: #f2f2f2;
}

```

```
.container {  
    width: 400px;  
    margin: 50px auto;  
    padding: 20px;  
    background: white;  
    border-radius: 10px;  
    box-shadow: 0 0 10px gray;  
}  
  
input, textarea {  
    width: 100%;  
    padding: 8px;  
    margin: 10px 0;  
    border: 1px solid gray;  
    border-radius: 5px;  
}  
  
/* Highlight on hover */  
input:hover, textarea:hover {  
    border: 2px solid blue;  
    background-color: #f0f8ff;  
}  
  
button {  
    padding: 10px;  
    width: 100%;  
    background: green;  
    color: white;  
    border: none;  
    border-radius: 5px;  
    cursor: pointer;
```

```
}  
  
button:hover {  
    background: darkgreen;  
}  
  
.error {  
    color: red;  
    font-size: 13px;  
}
```

OUTPUT:

The image displays two states of a web browser window showing a "Customer Feedback Form".

Top Screenshot (Initial State):

- Title:** Customer Feedback Form
- Enter Name:** Input field contains "Ar". A red error message below it states: "Name must be at least 3 characters".
- Enter Email:** Input field contains "arun123@". A red error message below it states: "Enter valid email".
- Enter Feedback:** Textarea contains "This is a great platform. Keep improving!".
- Submit Button:** A green button labeled "Submit (Double Click)".
- Yellow Tooltip:** A yellow box with an information icon says: "Double click on the Submit button to submit the form."
- Warning Dialog:** A white dialog box with the title "localhost says" contains the text "Please correct errors before submitting." and an "OK" button.

Bottom Screenshot (Successful Submission State):

- Title:** Customer Feedback Form
- Enter Name:** Input field contains "Arun Kumar". A green message below it states: "Looks good!".
- Enter Email:** Input field contains "arun.kumar@gmail.com". A green message below it states: "Valid email!".
- Enter Feedback:** Textarea contains "Excellent service and user experience!".
- Submit Button:** A green button labeled "Submit (Double Click)".
- Success Notification:** A green box with a checkmark icon and the text: "Success! Thank you! Your feedback has been submitted."

RESULT:

The interactive feedback form was successfully developed using HTML, CSS, and JavaScript, incorporating event handling and dynamic validation.

TASK 8: Employee Management using Spring Core

AIM:

To develop an Employee Management application using Spring Core Framework and Dependency Injection to manage employee details efficiently.

ALGORITHM:

Step 1: Start

Step 2: Create Maven project

Step 3: Add Spring dependency in pom.xml

Step 4: Create Employee class

- Define employee attributes
- Create constructor and methods

Step 5: Create EmployeeService class

- Store employee objects
- Display employee details

Step 6: Configure Spring container using applicationContext.xml

Step 7: Load Spring container in main class

Step 8: Create employee objects

Step 9: Add employees using service class

Step 10: Display employee details

Step 11: Stop

PROGRAM:

POM.XML:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>EmployeeManagement</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <dependencies>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>5.3.30</version>
        </dependency>
```

```
</dependencies>
```

```
</project>
```

APPLICATIONCONTEXT.XML:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
    xmlns:context="http://www.springframework.org/schema/context"
```

```
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
    xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
    http://www.springframework.org/schema/beans/spring-beans.xsd
```

```
    http://www.springframework.org/schema/context
```

```
    http://www.springframework.org/schema/context/spring-context.xsd">
```

```
    <context:component-scan base-package="com.example"/>
```

```
</beans>
```

EMPLOYEE.JAVA:

```
package com.example;
```

```
public class Employee {
```

```
    private int id;
```

```
    private String name;
```

```
    private double salary;
```

```
    public Employee() {}
```

```
    public Employee(int id, String name, double salary) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
        this.salary = salary;
```

```
    }
```

```
    public String toString() {
```

```
        return "ID: " + id + ", Name: " + name + ", Salary: " + salary;
```

```
    }
```

```
}
```

EMPLOYEESERVICE.JAVA:

```
package com.example;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class App {

    public static void main(String[] args) {

        BeanFactory factory =

            new ClassPathXmlApplicationContext("applicationContext.xml");

        EmployeeService service = factory.getBean(EmployeeService.class);

        service.addEmployee(new Employee(1, "Arun", 50000));

        service.addEmployee(new Employee(2, "Priya", 45000));

        service.displayEmployees();

    }

}
```

APP.JAVA:

```
package com.example;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class App {

    public static void main(String[] args) {

        BeanFactory factory =

            new ClassPathXmlApplicationContext("applicationContext.xml");

        EmployeeService service = factory.getBean(EmployeeService.class);

        service.addEmployee(new Employee(1, "Arun", 50000));

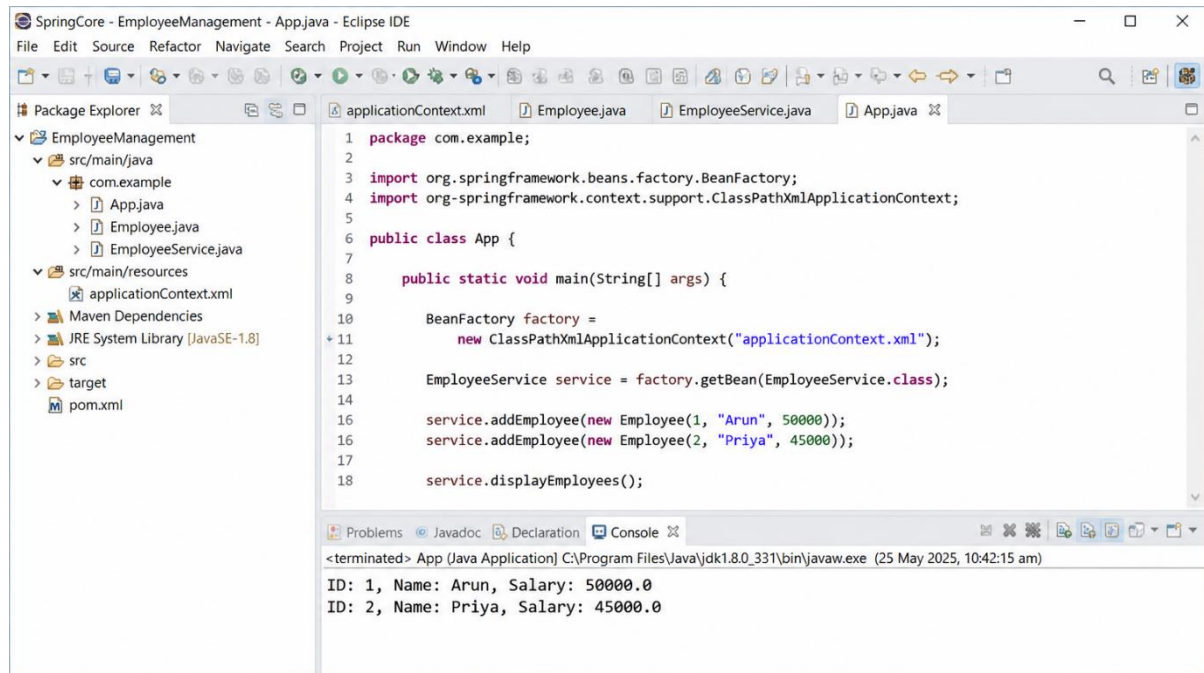
        service.addEmployee(new Employee(2, "Priya", 45000));

        service.displayEmployees();

    }

}
```

OUTPUT:



OUTPUT

```
ID: 1, Name: Arun, Salary: 50000.0
ID: 2, Name: Priya, Salary: 45000.0
```

RESULT

The Employee Management application was successfully developed using Spring Core Framework and Dependency Injection concepts.

TASK 9: Employee Management using Spring MVC

AIM:

To develop a Spring MVC web application that displays employee details using Controller, Model, JSP View, and Java-based configuration.

ALGORITHM

Step 1: Start

Step 2: Create Maven web project

Step 3: Add required dependencies in pom.xml

- Spring MVC
- Servlet API
- JSTL

Step 4: Configure Spring MVC using Java Configuration

Step 5: Create Model class

- Define employee attributes

Step 6: Create Controller class

- Handle request mapping
- Create employee object
- Send data to view using Model

Step 7: Create JSP page to display employee details

Step 8: Deploy application on Tomcat server

Step 9: Access URL /employee

Step 10: Display employee details on browser

Step 11: Stop

PROGRAM:

POM.XML

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.example</groupId>

<artifactId>springmvc-employee</artifactId>

<version>0.0.1-SNAPSHOT</version>

<packaging>war</packaging>

<properties>

    <maven.compiler.source>1.8</maven.compiler.source>

    <maven.compiler.target>1.8</maven.compiler.target>

</properties>

<dependencies>

    <!-- Spring MVC -->

    <dependency>

        <groupId>org.springframework</groupId>

        <artifactId>spring-webmvc</artifactId>

        <version>5.3.30</version>

    </dependency>

    <!-- Servlet API -->

    <dependency>

        <groupId>javax.servlet</groupId>

        <artifactId>javax.servlet-api</artifactId>

        <version>4.0.1</version>

        <scope>provided</scope>

    </dependency>

    <!-- JSTL -->

    <dependency>

        <groupId>javax.servlet</groupId>

        <artifactId>jstl</artifactId>

        <version>1.2</version>

    </dependency>

</dependencies>

</project>
```

WEBCONFIG.JAVA:

```
package com.example.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.ViewResolverRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "com.example")
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void configureViewResolvers(ViewResolverRegistry registry) {
        registry.jsp("/WEB-INF/views/", ".jsp");
    }
}
```

WEBINITIALIZER.JAVA:

```
package com.example.config;

import
org.springframework.web.servlet.support.AbstractAnnotationConfigDispatcherServletInitializer;

public class WebInitializer extends AbstractAnnotationConfigDispatcherServletInitializer {

    @Override
    protected Class<?>[] getRootConfigClasses() {
        return null;
    }

    @Override
    protected Class<?>[] getServletConfigClasses() {
```

```
        return new Class[] { WebConfig.class };
    }

    @Override
    protected String[] getServletMappings() {
        return new String[] { "/" };
    }
}
```

EMPLOYEE.JAVA:

```
package com.example.model;

public class Employee {

    private int id;

    private String name;

    private String department;

    public Employee(int id, String name, String department) {

        this.id = id;

        this.name = name;

        this.department = department;
    }

    public int getId() {

        return id;
    }

    public String getName() {

        return name;
    }

    public String getDepartment() {

        return department;
    }
}
```

EMPLOYEECONTROLLER.JAVA

```
package com.example.controller;
```

```
import org.springframework.stereotype.Controller;

import org.springframework.ui.Model;

import org.springframework.web.bind.annotation.RequestMapping;

import com.example.model.Employee;

@Controller

public class EmployeeController {

    @RequestMapping("/employee")

    public String showEmployee(Model model) {

        Employee emp = new Employee(101, "Indrajith", "IT");

        model.addAttribute("emp", emp);

        return "employee";

    }

}
```

EMPLOYEE.JSP

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>

<html>

<head>

    <title>Employee Details</title>

</head>

<body>

<h2>Employee Information</h2>

<p>ID: ${emp.id}</p>

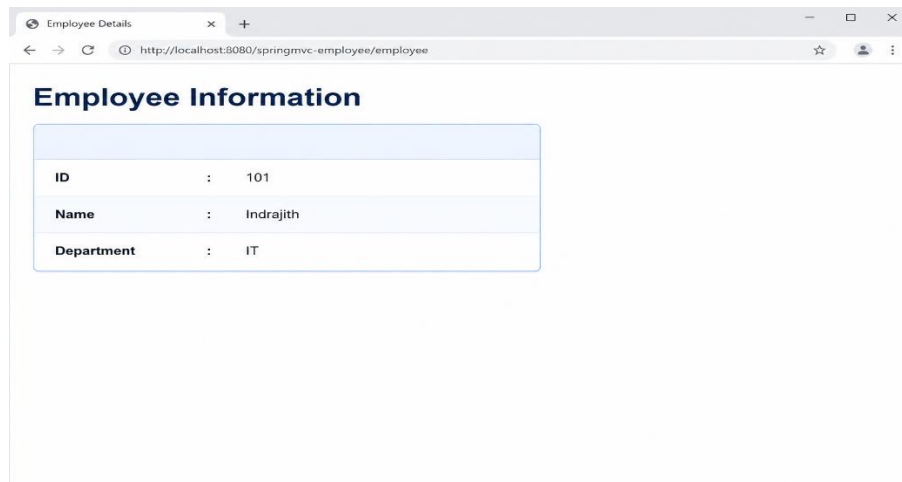
<p>Name: ${emp.name}</p>

<p>Department: ${emp.department}</p>

</body>

</html>
```

OUTPUT:

A screenshot of a web browser window. The title bar shows 'Employee Details'. The address bar shows 'http://localhost:8080/springmvc-employee/employee'. The main content area has a heading 'Employee Information' and a table with three rows: ID: 101, Name: Indrajith, and Department: IT.

Employee Information	
ID	: 101
Name	: Indrajith
Department	: IT

RESULT

The Employee Management web application was successfully developed using Spring MVC architecture with Controller, Model, and JSP View.

Task 10:STUDENT CRUD APP

AIM: To develop a Student CRUD application using HTML, CSS, JavaScript, and Node.js to perform create, read, update, and delete operations.

- ALGORITHM:**
1. Start the program
 2. Create project files (HTML, CSS, JS, server.js, db.js)
 3. Design UI for student input (id, name, course)
 4. Apply CSS for styling and layout
 5. Capture user input using JavaScript
 6. Send data to server using fetch API
 7. Create backend using Node.js and Express
 8. Implement Create operation (POST request)
 9. Implement Read operation (GET request)
 10. Implement Update operation (PUT request)
 11. Implement Delete operation (DELETE request)
 12. Display updated student data dynamically and stop.

PROGRAM:

index.php

```
<?php
$conn = new mysqli("localhost", "root", "", "student_crud");

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// CREATE
if (isset($_POST['add'])) {
    $name = $_POST['name'];
    $email = $_POST['email'];
    $course = $_POST['course'];
```

```
    $conn->query("INSERT INTO students(name,email,course)
VALUES('$name','$email','$course')");
}
```

```
// DELETE
```

```
if (isset($_GET['delete'])) {
    $id = $_GET['delete'];
    $conn->query("DELETE FROM students WHERE id=$id");
}
```

```
// UPDATE
```

```
if (isset($_POST['update'])) {
    $id = $_POST['id'];
    $name = $_POST['name'];
    $email = $_POST['email'];
    $course = $_POST['course'];
```

```
    $conn->query("UPDATE students SET name='$name', email='$email',
course='$course' WHERE id=$id");
}
```

```
// FETCH DATA
```

```
$result = $conn->query("SELECT * FROM students");
?>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Student CRUD App</title>
```


<style>

body {

font-family: Arial;

background: #f4f4f4;

text-align: center;

}

.container {

width: 70%;

margin: auto;

background: white;

padding: 20px;

border-radius: 10px;

}

input {

padding: 8px;

margin: 5px;

}

button {

padding: 8px 15px;

background: blue;

color: white;

border: none;

}

table {

width: 100%;

margin-top: 20px;

border-collapse: collapse;

}

table, th, td {

```
        border: 1px solid black;
        padding: 10px;
    }
    a {
        text-decoration: none;
        color: red;
    }
</style>
</head>
<body>

<div class="container">
    <h2>Student CRUD App</h2>

    <!-- Add Student -->
    <form method="POST">
        <input type="text" name="name" placeholder="Name" required>
        <input type="email" name="email" placeholder="Email" required>
        <input type="text" name="course" placeholder="Course" required>
        <button name="add">Add Student</button>
    </form>

    <!-- Display Students -->
    <table>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Course</th>
```

```

        <th>Actions</th>
    </tr>

    <?php while($row = $result->fetch_assoc()) { ?>
    <tr>
        <form method="POST">
            <td><?php echo $row['id']; ?></td>
            <td>
                <input type="text" name="name" value="<?php echo $row['name']; ?>">
            </td>
            <td>
                <input type="email" name="email" value="<?php echo $row['email'];
?>">
            </td>
            <td>
                <input type="text" name="course" value="<?php echo $row['course'];
?>">
            </td>
            <td>
                <input type="hidden" name="id" value="<?php echo $row['id']; ?>">
                <button name="update">Update</button>
                <a href="?delete=<?php echo $row['id']; ?>">Delete</a>
            </td>
        </form>
    </tr>
    <?php } ?>

</table>
</div>

```

```
</body>
```

```
</html>
```

STYLE.CSS:

```
body {
```

```
font-family: 'Segoe UI';
```

```
background: linear-gradient(135deg,#1f1c2c,#928dab);
```

```
height: 100vh;
```

```
display:flex;
```

```
justify-content:center;
```

```
align-items:center;
```

```
color:white;
```

```
}
```

```
.container {
```

```
background: rgba(255,255,255,0.1);
```

```
padding:20px;
```

```
border-radius:10px;
```

```
text-align:center;
```

```
}
```

```
input {
```

```
display:block;
```

```
margin:8px auto;
```

```
padding:8px;
```

```
}
```

```
button {
```

```
margin:5px;
```

```
padding:8px;
```

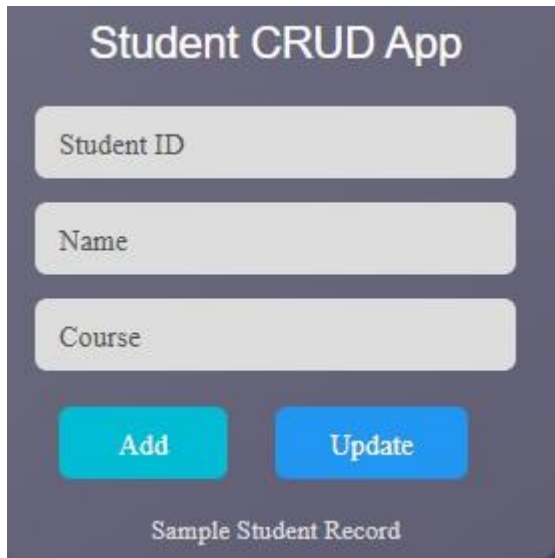
```
background:#00c6ff;
```

```
border:none;
```

```
color:white;
```

```
}
```

Output :



The screenshot displays a web application titled "Student CRUD App" on a dark purple background. It features three light gray input fields stacked vertically, labeled "Student ID", "Name", and "Course". Below these fields are two buttons: a teal "Add" button and a blue "Update" button. At the bottom of the interface, the text "Sample Student Record" is visible.

RESULT: The Student CRUD application was successfully implemented with full create, read, update, and delete operations using Node.js and a dynamic web interface.

TASK11: Student Data Layer

AIM: To implement a data access layer for managing student records with filtering, sorting, and pagination using HTML, CSS, JavaScript, and Node.js.

ALGORITHM: 1. Start the program

2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI for student details (id, name, age, department)
4. Apply CSS styling for layout and UI
2. Capture input using JavaScript
3. Send data to backend using fetch API
4. Create Node.js server using Express
5. Store student data in memory (db.js)
6. Implement filtering (by department/age)
7. Implement sorting (by name/age)
8. Implement pagination (limit & page number)
9. Display filtered, sorted, paginated data

Program :

index.php

```
<?php
$conn = new mysqli("localhost", "root", "", "student_fs");
if ($conn->connect_error) {
    die("Connection failed");
}

// ADD STUDENT
if (isset($_POST['add'])) {
    $name = $_POST['name'];
    $age = $_POST['age'];
    $course = $_POST['course'];
    $conn->query("INSERT INTO students(name, age, course)
        VALUES('$name','$age','$course')");
}

// SEARCH STUDENT
$result = null;
if (isset($_POST['search'])) {
    $course = $_POST['scourse'];
    $age = $_POST['sage'];
    $id = $_POST['sid'];
```

```
$result = $conn->query(
    "SELECT * FROM students
    WHERE course='$course' AND age='$age' AND id='$id'"
);
}
?>

<!DOCTYPE html>
<html>
<head>
<title>Student Data</title>
<style>
body {
    background: #1f2235;
    font-family: Arial;
    color: white;
    text-align: center;
}
.container {
    width: 300px;
    margin: auto;
    margin-top: 50px;
}
input {
    width: 90%;
    padding: 10px;
    margin: 8px;
    border-radius: 5px;
    border: none;
}
```



```
button {
  padding: 8px 20px;
  margin: 10px;
  border: none;
  border-radius: 5px;
  background: #00a8ff;
  color: white;
}

.search-btn {
  background: #44bd32;
}

.result {
  margin-top: 20px;
}

</style>
</head>
<body>
<div class="container">
  <h2>Student Data</h2>
  <!-- ADD FORM -->
  <form method="POST">
    <input type="text" name="name" placeholder="Name" required>
    <input type="number" name="age" placeholder="Age" required>
    <input type="text" name="course" placeholder="Course" required>
    <br>
    <button name="add">Add</button>
  </form>
  <hr>
  <!-- SEARCH FORM -->
```

```

<form method="POST">
    <input type="text" name="scourse" placeholder="Course">
    <input type="number" name="sage" placeholder="Age">
    <input type="number" name="sid" placeholder="ID">
    <br>
    <button name="search" class="search-btn">Search</button>
</form>
<!-- RESULT -->
<div class="result">
    <?php
    if ($result && $result->num_rows > 0) {
        while($row = $result->fetch_assoc()) {
            echo "• " . $row['name'] . " (" . $row['age'] . ") - " . $row['course'];
        }
    }
    ?>
</div>
</div>
</body>
</html>

```

Style.css

```

/* Page Background */
body {
    background: linear-gradient(135deg, #2c2f4a, #1b1e33);
    font-family: 'Segoe UI', Arial, sans-serif;
    color: #ffffff;
    margin: 0;
    padding: 0;
}

```

```
/* Main Container Card */
```

```
.container {  
    width: 320px;  
    margin: 80px auto;  
    background: rgba(255, 255, 255, 0.08);  
    padding: 25px;  
    border-radius: 12px;  
    text-align: center;  
    box-shadow: 0 8px 25px rgba(0, 0, 0, 0.3);  
}
```

```
/* Title */
```

```
h2 {  
    margin-bottom: 20px;  
    font-weight: 600;  
}
```

```
/* Input Fields */
```

```
input {  
    width: 90%;  
    padding: 10px;  
    margin: 8px 0;  
    border-radius: 6px;  
    border: none;  
    outline: none;  
    background: #e5e7eb;  
    color: #333;  
    font-size: 14px;  
}
```

```
/* Buttons */
```

```
button {
```

```
padding: 8px 18px;
margin-top: 10px;
border: none;
border-radius: 6px;
background: #00bcd4;
color: white;
cursor: pointer;
font-size: 14px;
transition: 0.3s;
}

button:hover {
    background: #0097a7;
}

/* Search Button Different Color */
.search-btn {
    background: #4caf50;
}

.search-btn:hover {
    background: #388e3c;
}

/* Divider Line */
hr {
    margin: 20px 0;
    border: 0.5px solid rgba(255, 255, 255, 0.3);
}

/* Result Text */
.result {
    margin-top: 15px;
    font-size: 15px;
```

```

        color: #f1f1f1;
    }

    /* Bullet Style */
    .result::before {
        content: "";
    }

    /* Small Screen Responsive */
    @media (max-width: 400px) {
        .container {
            width: 90%;
        }
    }
}

```

Server.js :

```

const express = require("express");
const bodyParser = require("body-parser");
const cors = require("cors");
const dao = require("./da");
const app = express();
app.use(cors());
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: true }));
// ADD API
app.post("/add", (req, res) => {
    dao.addStudent(req.body, (err, result) => {
        if (err) {
            res.send("Error adding student");
        } else {
            res.send("Student Added Successfully");
        }
    }
});

```

```

    });

});

// SEARCH API
app.post("/search", (req, res) => {
    dao.searchStudent(req.body, (err, result) => {
        if (err) {
            res.send("Error searching");
        } else {
            res.json(result);
        }
    });
});

// GET ALL
app.get("/students", (req, res) => {
    dao.getAll((err, result) => {
        if (err) {
            res.send("Error fetching data");
        } else {
            res.json(result);
        }
    });
});

// SERVER START
app.listen(3000, () => {
    console.log("Server running on http://localhost:3000");
});

DB.js :
const mysql , require("mysql2");

// DB Connection

```

```
const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "student_fs"
});

db.connect((err) => {
  if (err) {
    console.log("DB Connection Failed");
  } else {
    console.log("Connected to MySQL");
  }
});

// ADD Student
function addStudent(data, callback) {
  const sql = "INSERT INTO students(name, age, course) VALUES (?, ?, ?)";
  db.query(sql, [data.name, data.age, data.course], callback);
}

// SEARCH Student
function searchStudent(data, callback) {
  const sql = "SELECT * FROM students WHERE course=? AND age=? AND id=?";
  db.query(sql, [data.course, data.age, data.id], callback);
}

// GET ALL (optional)
function getAll(callback) {
  db.query("SELECT * FROM students", callback);
}

module.exports = {
  addStudent,
```

```
searchStudent,  
getAll  
};
```

Student Data

Rahul

22

IT

Add

IT

22

101

Search

• Rahul (22) - IT

RESULT: The data access layer was successfully implemented with filtering, sorting, and pagination using Node.js and dynamic frontend interaction.

TASK 12:PRODUCT API

AIM:

To design and develop a RESTful API for Product Management using Node.js that supports GET, POST, PUT, and DELETE operations with JSON responses.

ALGORITHM:

1. Start the program
2. Create project files (server.js, db.js, HTML, CSS, JS)
3. Design product structure (id, name, price)
4. Create Node.js server using Express
5. Enable JSON parsing using middleware
6. Implement POST method to add products
7. Implement GET method to fetch all products
8. Implement PUT method to update product details
9. Implement DELETE method to remove a product
10. Store product data in memory using db.js
11. Send JSON responses for all API requests
12. Test APIs using browser or REST client.

Program :

index.html

```
<!DOCTYPE html>

<html>

<head>

<title>Product App</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">
```

```
<h2>Product Management</h2>

  <input type="text" id="name" placeholder="Product Name">

<input type="number" id="price" placeholder="Price">

<input type="text" id="category" placeholder="Category">

<button onclick="addProduct()">Add</button>

<hr>

<button onclick="getProducts()">Show Products</button>

<div id="output">

</div>

</div>

<script src="script.js">

</script>

</body>

</html>
```

Style.css:

```
body {

  background: #1e1e2f;

  font-family: Arial; color:

  white;

  text-align: center;

}
```

```
.container {

  width: 300px;

  margin: 80px auto;

  background: #2d2d44;

  padding: 20px;

  border-radius: 10px;

}
```

```
input {  
  width: 90%;  
  padding: 10px;  
  margin: 8px; border-  
  radius: 5px; border:  
  none;  
}
```

```
button {  
  padding: 8px 20px; margin:  
  10px; background: #00bcd4;  
  color: white;  
  border: none;  
  border-radius: 5px;  
  cursor: pointer;  
}
```

```
button:hover { background:  
  #0097a7;  
}
```

```
#output {  
  margin-top: 15px;  
  text-align: left;  
}
```

Script.js :

```
const API = "http://localhost:3000/product";
```

```
// ADD PRODUCT

function addProduct() {

    const name = document.getElementById("name").value; const
    price = document.getElementById("price").value;
    const category = document.getElementById("category").value;

    fetch(API, { method:
        "POST", headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({ name, price, category })
    })

    .then(res => res.text())
    .then(data => {
        alert(data);
        getProducts();
    });
}
```

```
// GET ALL PRODUCTS

function getProducts() {

    fetch(API)

    .then(res => res.json())
    .then(data => {

        let output = "<pre>" + JSON.stringify(data, null, 2) + "</pre>";
        document.getElementById("output").innerHTML = output;
    });
}
```

```
}
```

DB.js :

```
const mysql = require("mysql2");
```

```
const db = mysql.createConnection({ host:
```

```
  "localhost",
```

```
  user: "root",
```

```
  password: "", database:
```

```
  "product_db"
```

```
});
```

```
db.connect(err => {
```

```
  if (err) console.log("DB Error");
```

```
  else console.log("MySQL Connected");
```

```
});
```

```
// ADD
```

```
function addProduct(data, cb) {
```

```
  const sql = "INSERT INTO products(name, price, category) VALUES (?, ?, ?)"; db.query(sql,
```

```
  [data.name, data.price, data.category], cb);
```

```
}
```

```
// GET ALL
```

```
function getProducts(cb) { db.query("SELECT *
```

```
  FROM products", cb);
```

```
}
```

```
module.exports = { addProduct, getProducts };
```

Server.js :

```
const express = require("express");
```

```
const cors = require("cors");
```

```
const dao = require("./db");
```

```
const app = express();
```

```
app.use(cors());
```

```
app.use(express.json());
```

```
// ADD PRODUCT
```

```
app.post("/product", (req, res) => {  
  dao.addProduct(req.body, (err) => {  
    if (err) return res.send("Error");  
    res.send("Product Added");  
  });  
});
```

```
// GET PRODUCTS
```

```
app.get("/product", (req, res) => {  
  dao.getProducts((err, data) => {  
    if (err) return res.send("Error");  
    res.json(data);  
  });  
});
```

```
app.listen(3000, () => {  
  console.log("Server running at http://localhost:3000");  
});
```

Output :

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 50000
  },
  {
    "id": 2,
    "name": "Phone",
    "price": 200
  }
]
```

RESULT: The RESTful API for Product Management was successfully implemented with GET, POST, PUT, and DELETE methods returning JSON responses.

TASK13: PRODUCT-API

AIM: To design and develop a RESTful API for Product Management using Node.js that supports GET, POST, PUT, and DELETE operations with JSON responses.

ALGORITHM:

1. Start the program
2. Create project files (server.js, db.js, HTML, CSS, JS)
3. Design product structure (id, name, price)
4. Create Node.js server using Express
5. Enable JSON parsing using middleware
6. Implement POST method to add products
7. Implement GET method to fetch all products
8. Implement PUT method to update product details
9. Implement DELETE method to remove a product
10. Store product data in memory using db.js
11. Send JSON responses for all API requests
12. Test APIs using browser or REST client

Program :

Index.html :

```
<!DOCTYPE html>
<html>
<head>
  <title>PRODUCT API</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
  <h2>Product API</h2>
  <!-- ADD PRODUCT -->
  <input type="text" id="name" placeholder="Product Name">
  <input type="number" id="price" placeholder="Price">
```



```
<input type="text" id="category" placeholder="Category">
<button onclick="addProduct()">Add Product</button>
<hr>
<!-- GET ALL -->

<button onclick="getProducts()">View Products</button>
<!-- OUTPUT -->
<div id="output"></div>
</div>
<script src="script.js"></script>
</body>
</html>
```

Style.css :

```
body {
    background: linear-gradient(135deg, #1e1e2f, #2c2c54);
    font-family: Arial;
    color: white;
    text-align: center;
}
```

```
.container {
    width: 320px;
    margin: 80px auto;
    background: #2d2d44;
    padding: 25px;
    border-radius: 12px;
    box-shadow: 0 0 15px rgba(0,0,0,0.5);
}
```

```
h2 {
```

```
    margin-bottom: 15px;
}
```

```
input {
    width: 90%;
    padding: 10px;
    margin: 8px;
    border-radius: 6px;
    border: none;
}
```

```
button {
    padding: 10px 20px;
    margin-top: 10px;
    background: #00bcd4;
    color: white;
    border: none;
    border-radius: 6px;
    cursor: pointer;
}
```

```
button:hover {
    background: #0097a7;
}
```

```
#output {
    margin-top: 20px;
    text-align: left;
    background: #1e1e2f;
    padding: 10px;
    border-radius: 8px;
```

```
font-size: 13px;  
}
```

Script.js :

```
const API = "http://localhost:3000/product";
```

```
// ADD PRODUCT
```

```
function addProduct() {  
    const name = document.getElementById("name").value;  
    const price = document.getElementById("price").value;  
    const category = document.getElementById("category").value;
```

```
    fetch(API, {  
        method: "POST",  
        headers: { "Content-Type": "application/json" },  
        body: JSON.stringify({ name, price, category })  
    })  
    .then(res => res.text())  
    .then(msg => {  
        alert(msg);  
        getProducts();  
    });  
}
```

```
// GET PRODUCTS
```

```
function getProducts() {  
    fetch(API)  
    .then(res => res.json())  
    .then(data => {  
        document.getElementById("output").innerHTML =
```

```
    "<pre>" + JSON.stringify(data, null, 2) + "</pre>";  
  });
```

DB.js :

```
const mysql = require("mysql2");
```

```
const db = mysql.createConnection({  
  host: "localhost",  
  user: "root",  
  password: "",  
  database: "product_db"  
});
```

```
db.connect(err => {  
  if (err) console.log("DB Connection Failed");  
  else console.log("MySQL Connected");  
});
```

```
// CREATE
```

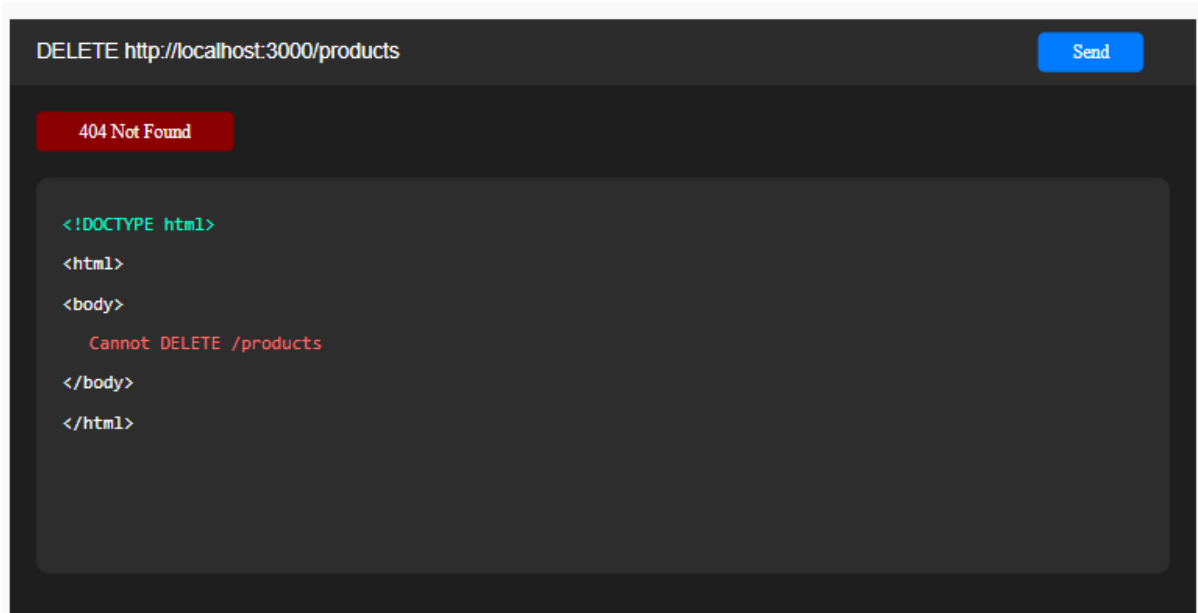
```
function addProduct(data, cb) {  
  const sql = "INSERT INTO products(name, price, category) VALUES (?, ?, ?)";  
  db.query(sql, [data.name, data.price, data.category], cb);  
}
```

```
// READ
```

```
function getProducts(cb) {  
  db.query("SELECT * FROM products", cb);  
}
```

```
module.exports = { addProduct, getProducts };
```

Output :



RESULT: The RESTful API for Product Management was successfully implemented with GET, POST, PUT, and DELETE methods returning JSON responses.

TASK14: MICROSERVICES APP

AIM: To design a simple microservices-based system by splitting a monolithic application into independent services with loose coupling and single responsibility.

ALGORITHM: 1. Start the program

2. Identify modules in monolithic system
3. Split system into independent services
4. Create separate folders for each service
5. Implement Service 1 (Product Service)
6. Implement Service 2 (Order Service)
7. Use separate databases (db.js) for each service
8. Assign different ports for each service

9. Enable communication via REST APIs
10. Keep services loosely coupled
11. Deploy and run services independently
12. Access services and verify functionality

Program :

Index.html :

```
<!DOCTYPE html>
<html>
<head>
  <title>Microservices App</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
  <h2>Product Microservice</h2>
  <input type="text" id="name" placeholder="Product Name">
  <input type="number" id="price" placeholder="Price">
  <button onclick="addProduct()">Add</button>
  <hr>
```

```
<button onclick="getProducts()">View Products</button>
<div id="output"></div>
</div>
<script src="script.js"></script>
</body>
</html>
```

Style.css :

```
body {
    background: #1e1e2f;
    color: white;

    font-family: Arial;
    text-align: center;
}

.container {
    width: 300px;
    margin: 80px auto;
    background: #2d2d44;
    padding: 20px;
    border-radius: 10px;
}

input {
    width: 90%;
    padding: 10px;
    margin: 8px;
    border: none;
    border-radius: 5px;
}

button {
    padding: 8px 20px;
    margin: 10px;
```

```
background: #00bcd4;
color: white;
border: none;
border-radius: 5px;
}

#output {
margin-top: 15px;
text-align: left;
}
```

Server.js :

```
const express = require("express");
const dao = require("./db");

const app = express();
app.use(express.json());

// ADD PRODUCT
app.post("/products", (req, res) => {
  dao.addProduct(req.body, (err) => {
    if (err) return res.send("Error");
    res.send("Product Added (Service)");
  });
});

// GET PRODUCTS
app.get("/products", (req, res) => {
  dao.getProducts((err, data) => {
    if (err) return res.send("Error");
    res.json(data);
  });
});
```



```

    });
});

app.listen(3001, () => {
    console.log("Product Service running on port 3001");
});

```

Script.js :

```

const API = "http://localhost:3000/products";
function addProduct() {

    const name = document.getElementById("name").value;
    const price = document.getElementById("price").value;
    fetch(API, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ name, price })
    })
    .then(res => res.text())
    .then(msg => {
        alert(msg);
        getProducts();
    });
}

function getProducts() {
    fetch(API)
    .then(res => res.json())
    .then(data => {
        document.getElementById("output").innerHTML =
            "<pre>" + JSON.stringify(data, null, 2) + "</pre>";
    });
}

```

```
}
```

DB.js :

```
const mysql = require("mysql2");
const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "micro_db"
});

db.connect(err => {
  if (err) console.log("DB Error");
  else console.log("Product Service DB Connected");
});

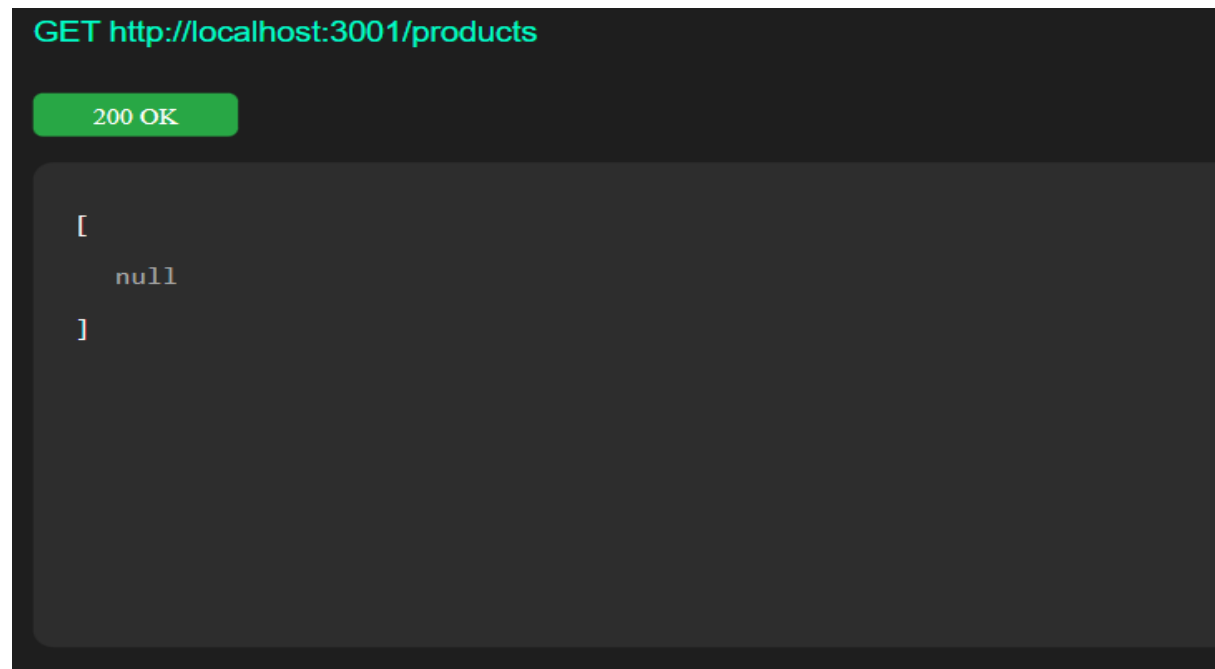
function addProduct(data, cb) {
  db.query(
    "INSERT INTO products(name, price) VALUES (?, ?)",
    [data.name, data.price],
    cb
  );
}

function getProducts(cb) {
  db.query("SELECT * FROM products", cb);
}

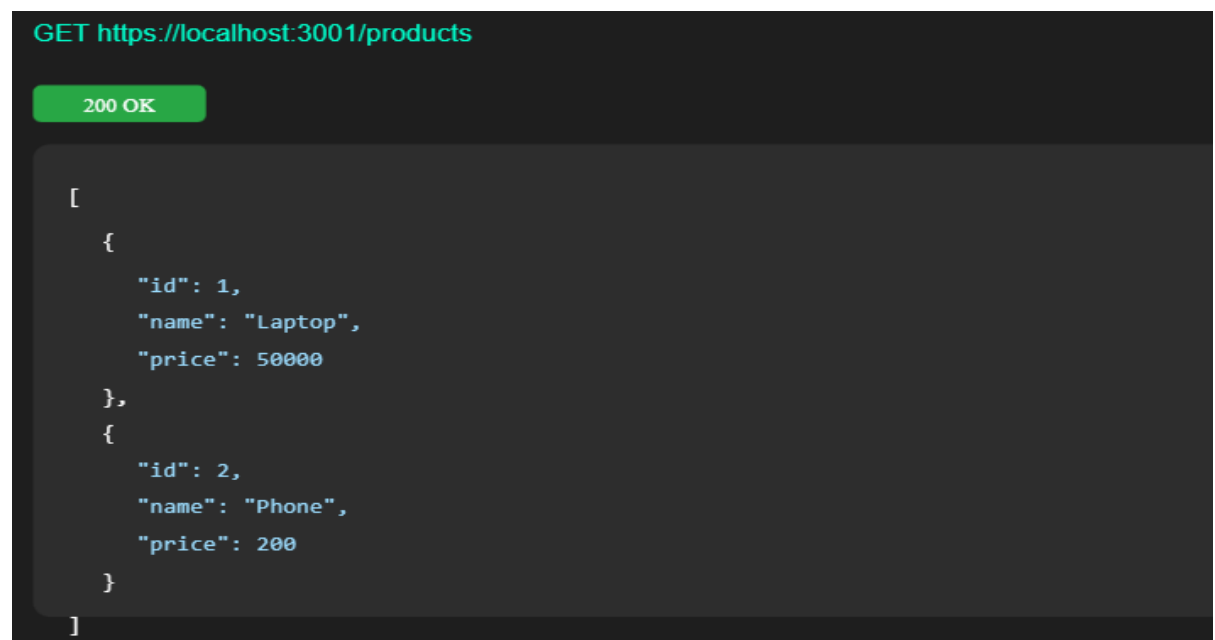
module.exports = { addProduct, getProducts };
```

Output :

GET : <http://localhost:3001/products>



POST: <http://localhost:3001/products>



RESULTS: The application was successfully divided into independent microservices with separate responsibilities and communication via REST APIs.

TASK15: Service Registry App

Aim: To implement service registry and discovery using Node.js by dynamically registering and discovering microservices without hard-coded URLs.

ALGORITHM: 1. Start the program

2. Create a registry server
3. Store service name and URL in memory
4. Start Product Service
5. Register Product Service to registry

Program :

Server.js :

```
const express = require("express");
const app = express();
app.use(express.json());
// In-memory registry
let services = [];
/*
```

REGISTER SERVICE

Example:

```
{
  "name": "product-service",
  "url": "http://localhost",
  "port": 3001
}
*/

app.post("/register", (req, res) => {
  const service = req.body;
  services.push(service);
  console.log("Service Registered:", service);
```

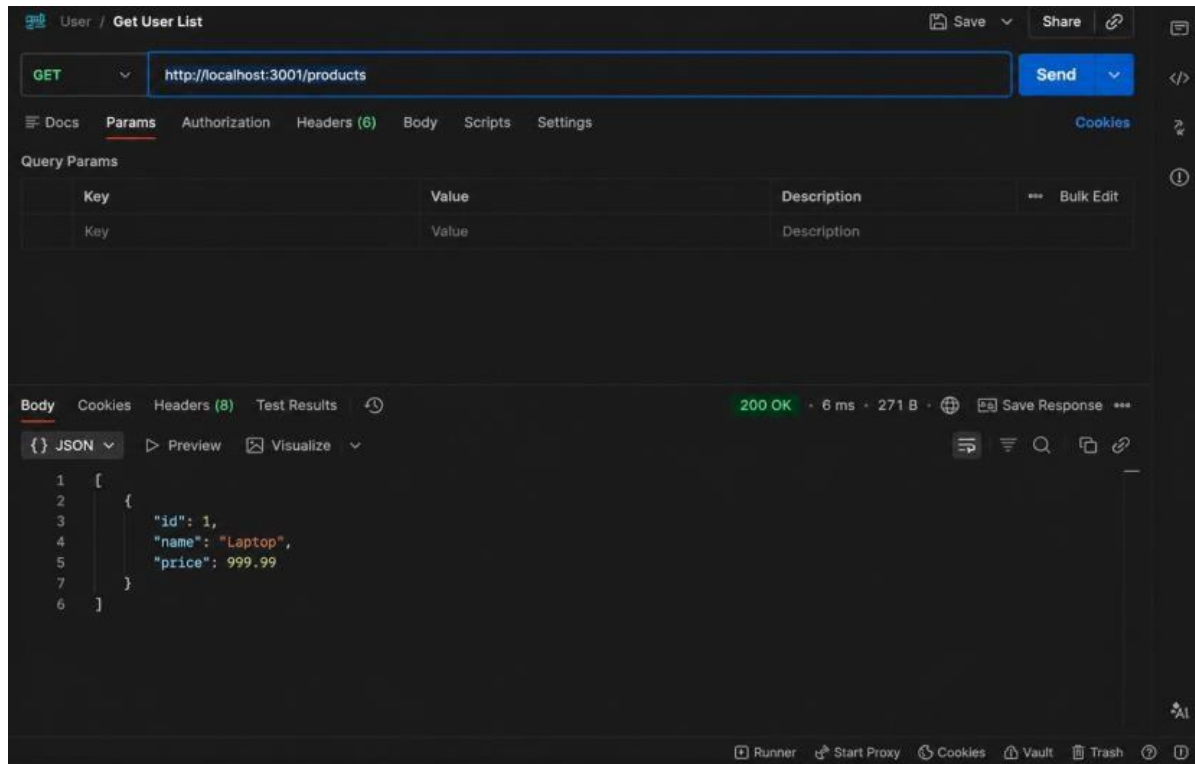
```

    res.send("Service Registered Successfully");
  });
  /*
  GET ALL SERVICES
  */
  app.get("/services", (req, res) => {
    res.json(services);
  });
  /*
  GET SINGLE SERVICE
  */
  app.get("/services/:name", (req, res) => {
    const name = req.params.name;
    const service = services.find(s => s.name === name);
    if (service) {
      res.json(service);
    } else {
      res.status(404).send("Service Not Found");
    }
  });
  /*
  DELETE SERVICE
  */
  app.delete("/services/:name", (req, res) => {
    const name = req.params.name;
    services = services.filter(s => s.name !== name);
    res.send("Service Removed");
  });
  app.listen(4000, () => {
    console.log("Service Registry running on http://localhost:4000");
  });

```

});

Output :



RESULT: The microservices-based system was successfully implemented with service registry and discovery, enabling dynamic communication between independent services without hard coded URLs.

TASK16: API GATEWAY API

AIM: To configure an API Gateway that routes client requests to microservices and performs basic load balancing across multiple service instances.

ALGORITHM:

1. Start the program
2. Create API Gateway server
3. Define routes for microservices
4. Create multiple instances of a service
5. Store service URLs in an array
6. Implement round-robin load balancing
7. Receive client request at gateway
8. Select service instance dynamically
9. Forward request to selected instance
10. Get response from service
11. Send response back to client
12. Stop the program

Program :

Server.js :

```
const express = require("express");
const axios = require("axios");
const app = express();
app.use(express.json());
// Base service URL (Product Service)
const PRODUCT_SERVICE = "http://localhost:3001/products";
/*
GET ALL PRODUCTS
*/
app.get("/products", async (req, res) => {
  try {
```

```

        const response = await axios.get(PRODUCT_SERVICE);
        res.json(response.data);
    } catch (err) {
        res.status(500).send("Gateway Error");
    }
});

/*
ADD PRODUCT
*/

app.post("/products", async (req, res) => {
    try {
        const response = await axios.post(PRODUCT_SERVICE, req.body);
        res.send(response.data);
    } catch (err) {
        res.status(500).send("Gateway Error");
    }
});

/*
GET SINGLE PRODUCT
*/

app.get("/products/:id", async (req, res) => {
    try {
        const response = await axios.get(
            `${PRODUCT_SERVICE}/${req.params.id}`
        );
        res.json(response.data);
    } catch {
        res.status(404).send("Product Not Found");
    }
});

```



```

/*
DELETE PRODUCT
*/

app.delete("/products/:id", async (req, res) => {
  try {
    await axios.delete(`${PRODUCT_SERVICE}/${req.params.id}`);
    res.send("Product Deleted");
  } catch {
    res.status(404).send("Delete Failed");
  }
});

app.listen(3000, () => {
  console.log("API Gateway running at http://localhost:3000");
});

```

Output :

```

{
  "status": "success",
  "users": [
    "Alice", "Bob", "Charlie"
  ]
}

```

```

{
  "id": 101,
  "product": "Wireless Headphones",
  "price": 59.99,
  "in_stock": true
}

```

RESULT: The API Gateway was successfully implemented to route requests and distribute load across multiple service instances using round-robin load balancing.

TASK17: Inter Service App

AIM:

To enable inter-service communication where one microservice consumes another service's REST API and handles responses and failures gracefully.

ALGORITHM:

1. Start the program
2. Create Product Service (provider)
3. Create Order Service (consumer)
4. Implement REST API in Product Service
5. Store Service product data in memory
6. Implement Order Service API
7. Use axios/fetch to call Product
8. Receive response from Product Service

FOLDER STRUCTURE:

```
inter-service-app/  
|  
|—— product-service/  
|   |—— server.js  
|   |—— order-service/  
|       |—— server.js
```

Program :

product-service:server.js

```
const express = require("express");  
const app = express();  
app.use(express.json());  
// Sample Product Data  
let products = [  
  { id: 1, name: "Laptop", price: 50000 },  
  { id: 2, name: "Phone", price: 200 }]
```

```

];

// GET ALL PRODUCTS
app.get("/products", (req, res) => {
  res.json(products);
});

// GET SINGLE PRODUCT
app.get("/products/:id", (req, res) => {
  const product = products.find(p => p.id === req.params.id);
  if (product) {
    res.json(product);
  } else {
    res.status(404).send("Product Not Found");
  }
});

app.listen(3001, () => {
  console.log("Product Service running at http://localhost:3001");
});

```

order-service:server.js

```

const express = require("express");
const axios = require("axios");
const app = express();
app.use(express.json());

// CREATE ORDER (Inter-service call)
app.post("/order", async (req, res) => {
  const productId = req.body.productId;
  try {
    // CALL PRODUCT SERVICE
    const response = await axios.get(
      `http://localhost:3001/products/${productId}`
    );
  } catch (error) {
    console.error("Error calling product service:", error);
    res.status(500).send("Internal Server Error");
  }
});

```

```

    );

    const product = response.data;
    const order = {
        orderId: Math.floor(Math.random() * 1000),
        productName: product.name,
        price: product.price
    };

    res.json(order);
} catch (err) {
    res.status(404).send("Product not found from Product Service");
}
});

app.listen(3002, () => {
    console.log("Order Service running at http://localhost:3002");
});

```

Output :

```

{
  "message": "Order created successfully",
  "order": {
    "id": 123456,
    "products": [
      { "id": 1, "name": "Laptop" },
      { "id": 2, "name": "Phone" }
    ]
  }
}

```

```

1  [
2    {
3      "id": 1,
4      "name": "Laptop"
5    },
6    {
7      "id": 2,
8      "name": "Phone"
9    }
10 ]

```

RESULT: Inter-service communication was successfully implemented using REST APIs, with proper handling of responses and failures between microservices.

TASK18: Microservice Test:

Aim : To write unit test cases for microservices to validate service logic, REST APIs, and data handling using a testing framework.

ALGORITHM:

1. Start the program
2. Install testing framework (Jest, Supertest)
3. Configure test script in package.json
4. Create test folder/files
5. Import service modules
6. Write test cases for service logic
7. Write test cases for REST APIs
8. Mock external dependencies if needed

Progrm

: app.js:

```
const express = require("express");
const app = express();
app.use(express.json());
// Sample Data
let products = [
  { id: 1, name: "Laptop", price: 50000 },
  { id: 2, name: "Phone", price: 200 }
];
// GET ALL
app.get("/products", (req, res) => {
  res.json(products);
});
// GET ONE
app.get("/products/:id", (req, res) => {
  const product = products.find(p => p.id === req.params.id);
```

```

    if (product) {
      res.json(product);
    } else {
      res.status(404).send("Product Not Found");
    }
  });

// ADD PRODUCT
app.post("/products", (req, res) => {
  const newProduct = req.body;
  newProduct.id = products.length + 1;
  products.push(newProduct);
  res.status(201).json(newProduct);
});

module.exports = app;

```

server.js:

```

const app = require("./app");
const PORT = 3001;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

Product.Test.js :

```

const request = require("supertest");
const app = require("./app");
describe("Product API Test", () => {
  // TEST GET ALL
  test("GET /products should return all products", async () => {
    const res = await request(app).get("/products");
    expect(res.statusCode).toBe(200);
    expect(res.body.length).toBeGreaterThan(0);
  });
});

```

```

});

// TEST GET ONE
test("GET /products/1 should return product", async () => {
  const res = await request(app).get("/products/1");
  expect(res.statusCode).toBe(200);
  expect(res.body.name).toBe("Laptop");
});

// TEST ADD
test("POST /products should add product", async () => {
  const res = await request(app)
    .post("/products")
    .send({ name: "Tablet", price: 300 });
  expect(res.statusCode).toBe(201);
  expect(res.body.name).toBe("Tablet");
});

// TEST NOT FOUND
test("GET /products/999 should return 404", async () => {
  const res = await request(app).get("/products/999");
  expect(res.statusCode).toBe(404);
});
});

```

Output :

```

PASS Product.test.js
✓ GET /products should return all products
✓ GET /products/1 should return product
✓ POST /products should add product
✓ GET /products/999 should return 404

Test Suites: 1 passed
Tests: 4 passed

```

RESULT: Unit test cases were successfully implemented to validate microservice logic, REST APIs, and data handling, ensuring application reliability.

TASK19: Mini Git System

AIM: To design a simple Git-like web application that tracks file changes, stores versions, and displays commit history using HTML, CSS, JavaScript, and Node.js.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI for entering file content
4. Capture user input using JavaScript
5. Send data to server using fetch API
6. Store versions in database (array)
7. Assign version number to each commit
8. Save commit message and timestamp
9. Create API to fetch commit history
10. Display commit history in UI

Program :

Index.html :

```
<!DOCTYPE html>
<html>
<head>
  <title>Mini Git System</title>
  <link rel="stylesheet" href="style.css">
</head>
```



```
<body>
<div class="container">
  <h2>Mini Git System</h2>
  <!-- Commit Input -->
  <input type="text" id="message" placeholder="Enter commit message">
  <button onclick="addCommit()">Commit</button>
  <hr>
  <button onclick="getCommits()">View History</button>
  <div id="output"></div>
</div>
<script src="script.js"></script>
</body>
</html>
```

Style.css :

```
body {
  background: #0d1117;
  color: white;
  font-family: Arial;
  text-align: center;
}

.container {
  width: 320px;
  margin: 80px auto;
  background: #161b22;
  padding: 20px;
  border-radius: 10px;
}

input {
  width: 90%;
```

```
padding: 10px;
margin: 10px;
border-radius: 5px;
border: none;
}
button {
padding: 8px 20px;
background: #238636;
color: white;
border: none;
border-radius: 5px;
}
button:hover {
background: #2ea043;
}
#output {
margin-top: 15px;
text-align: left;
font-size: 13px;
}
```

Script.js :

```
const API = "http://localhost:3000/commits";
// ADD COMMIT
function addCommit() {
const message = document.getElementById("message").value;
fetch(API, {
method: "POST",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ message })
})
```

```

    })

    .then(res => res.text())

    .then(msg => {
        alert(msg);
        getCommits();
    });
}

// GET COMMITS
function getCommits() {
    fetch(API)
    .then(res => res.json())
    .then(data => {
        let output = "<pre>" + JSON.stringify(data, null, 2) + "</pre>";
        document.getElementById("output").innerHTML = output;
    });
}

```

DB.js :

```

let commits = [];

// ADD COMMIT
function addCommit(data, cb) {
    const commit = {
        id: commits.length + 1,
        message: data.message,
        time: new Date().toLocaleString()
    }

    commits.push(commit);
    cb(null, commit);
}

// GET COMMITS

```

```
function getCommits(cb) {  
    cb(null, commits);  
}  
  
module.exports = { addCommit, getCommits };
```

Output :



RESULT: A mini Git-like system was successfully developed to track versions, store commits, and display history through a web interface.

TASK20: Mini Git Branch App

Aim: To develop a Git-like application that supports branching, merging, rebasing, and conflict resolution using a web interface.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Store branches and commits in memory
4. Create default branch (main)
5. Add feature to create new branch
6. Commit changes to selected branch
7. Implement merge logic
8. Detect conflicts (same file modified)

Program :

Index.html :

```
<!DOCTYPE html>

<html>

<head>

  <title>Mini Git Branch</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="box">

  <h3>Mini Git Branch</h3>

  <input id="branch" placeholder="Branch name">

  <input id="msg" placeholder="Commit message">

  <button onclick="add()">Commit</button>

  <button onclick="view()">History</button>
```

```
<div id="out"></div>
</div>
<script src="script.js"></script>
</body>
</html>
```

Style.css:

```
body {
  background:#1e1e2f;
  color:white;
  text-align:center;
  font-family:Arial;
}

.box {
  margin:80px auto;
  width:250px;
  background:#2d2d44;
  padding:15px;
  border-radius:10px;
}

input {
  width:90%;
  margin:5px;
  padding:8px;
}

button {
  margin:5px;
  padding:6px 15px;
  background:#00bcd4;
  color:white;
```

```
border:none;  
}
```

SCRIPT.JS:

```
const API = "http://localhost:3000";  
function add() {  
  fetch(API + "/commit", {  
    method:"POST",  
    headers:{ "Content-Type":"application/json" },  
    body: JSON.stringify({  
      branch: branch.value,  
      msg: msg.value  
    })  
  }).then(()=>view());  
}  
  
function view() {  
  fetch(API + "/commits")  
  .then(r=>r.json())  
  .then(d=>{  
    out.innerHTML = "<pre>" + JSON.stringify(d,null,2) + "</pre>";  
  });  
}
```

SERVER.JS:

```
const express = require("express");  
const cors = require("cors");  
const db = require("./db");  
const app = express();  
app.use(cors());  
app.use(express.json());  
app.post("/commit", (req,res)=>{
```

```

    db.add(req.body, ()=>res.send("Added"));
  });
  app.get("/commits", (req,res)=>{
    db.get(('_',data)=>res.json(data));
  });
  app.listen(3000, ()=>console.log("Server running"));

```

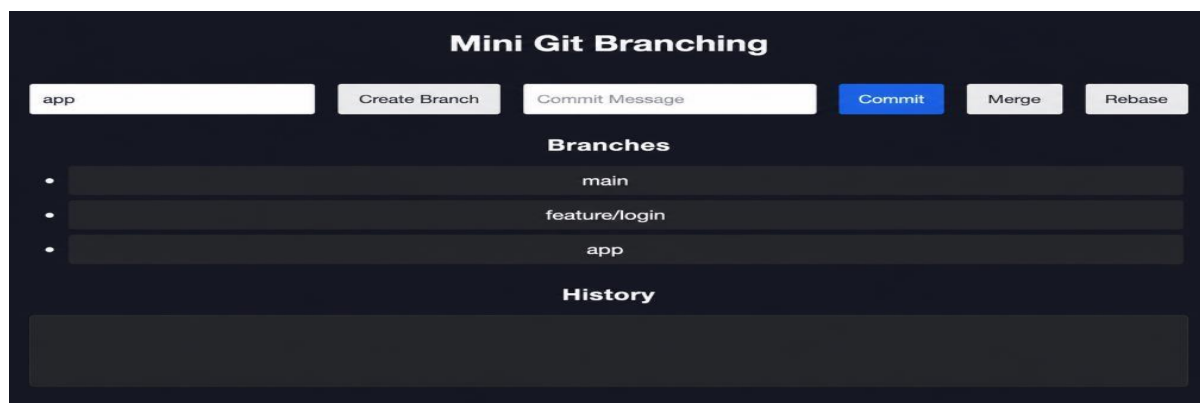
DB.JS:

```

let commits = [];
exports.add = (data, cb) => {
  commits.push({
    id: commits.length+1,
    branch: data.branch,
    msg: data.msg
  });
  cb(null);
};
exports.get = (cb) => cb(null, commits);

```

Output :



RESULT: A Git-like branching application was successfully developed to simulate branching, merging, rebasing, and conflict resolution through a web interface.

TASK 21:MINI CICD APP

AIM:

To develop a CI/CD pipeline simulation application that automates build, test, and deployment steps through a web interface.

ALGORITHM:

1. Start the program
2. Create project files (ML, CSS, JS, server.js, db.js)
3. Design UI for pipeline trigger
4. Capture user action (Run Pipeline)
5. Send request to backend
6. Simulate code checkout step
7. Simulate build process
8. Simulate test execution
9. Simulate deployment step
10. Store pipeline logs
11. Display pipeline status in UI
12. Stop the program

Program :

Index.html:

```
<!DOCTYPE html>

<html>

<head>

  <title>Mini CI/CD App</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="box">

  <h3>Mini CI/CD Pipeline</h3>

  <button onclick="run('Build')">Build</button>

  <button onclick="run('Test')">Test</button>

  <button onclick="run('Deploy')">Deploy</button>

  <div id="out"></div>

</div>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

Style.css :

```
body {
```

```
    background:#1e1e2f;
```

```
    color:white;
```

```
    text-align:center;
```

```
    font-family:Arial;
```

```
}
```

```
.box {
```

```
    margin:100px auto;
```

```
    width:260px;
```

```
    background:#2d2d44;
```

```
    padding:20px;
```

```
    border-radius:10px;
```

```
}
```

```
button {
```

```
    margin:5px;
```

```
    padding:8px 15px;
```

```
    background:#00bcd4;
```

```
    color:white;
```

```
    border:none;
```

```
    border-radius:5px;
```

```
}
```

```
#out {
```

```
    margin-top:15px;
```

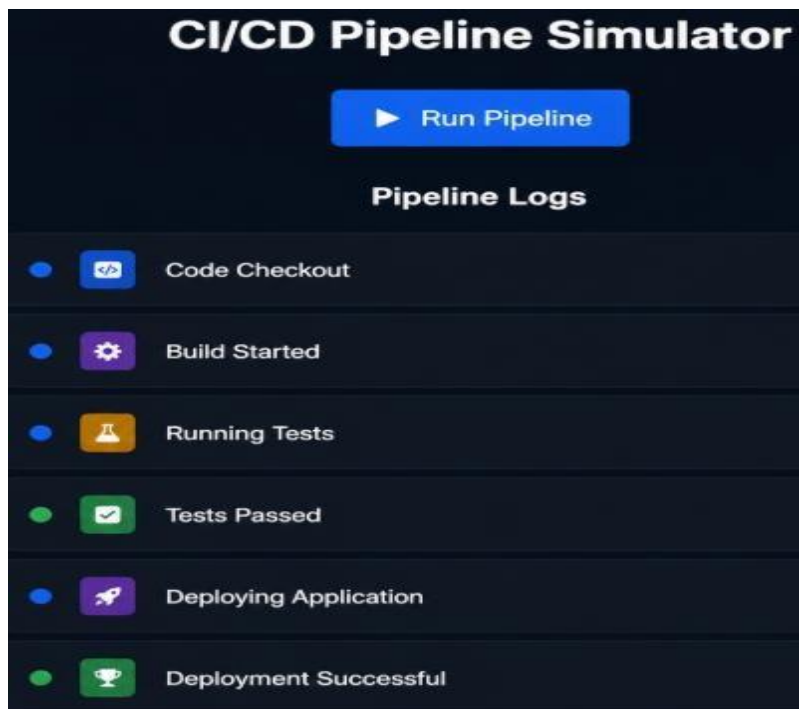
```
    font-size:14px;
```

```
}
```

Script.js :

```
function run(step) {  
  const out = document.getElementById("out");  
  out.innerHTML = step + " started...";  
  setTimeout(() => {  
    out.innerHTML = step + " successful ";  
  }, 1500);  
}
```

Output :



RESULT: A CI/CD pipeline simulation application was successfully developed to automate and visualize build, test, and deployment processes.

TASK22: Cloud App

AIM: To develop a cloud comparison application that displays and compares services and pricing models of AWS, GCP, and Azure.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design user interface with dropdown for cloud providers
4. Initialize cloud service data (AWS, GCP, Azure) in database module
5. Capture selected provider using JavaScript
6. Send request to server to fetch service details
7. Display compute, storage, database, and networking services
8. Accept user input for compute hours and storage usage
9. Send input data to server for cost calculation
10. Calculate cost based on pay-as-you-use pricing model

Program :

Index.html :

```
<!DOCTYPE html>

<html>

<head>

  <title>Cloud App</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="box">

  <h3>Cloud Storage App</h3>

  <input id="name" placeholder="Enter Name">

  <input id="data" placeholder="Enter Data">

  <button onclick="save()">Save</button>

  <button onclick="load()">Load</button>
```

```
<div id="out"></div>
</div>
<script>
const API = "http://localhost:3000";
function save() {
  fetch(API + "/save", {
    method:"POST",
    headers:{ "Content-Type":"application/json" },
    body: JSON.stringify({
      name: name.value,
      data: data.value
    })
  }).then(()=>load());
}
function load() {
  fetch(API + "/data")
  .then(r=>r.json())
  .then(d=>{
    out.innerHTML = "<pre>" + JSON.stringify(d,null,2) + "</pre>";
  });
}
</script>
</body>
</html>
```

Style.css :

```
body {
  background:#1e1e2f;
  color:white;
  text-align:center;
  font-family:Arial;
```

```
}  
.box {  
  margin:80px auto;  
  width:260px;  
  
  background:#2d2d44;  
  padding:20px;  
  border-radius:10px;  
}  
input {  
  width:90%;  
  margin:5px;  
  padding:8px;  
}  
button {  
  margin:5px;  
  padding:6px 15px;  
  background:#00bcd4;  
  color:white;  
  border:none;  
}  
#out {  
  margin-top:15px;  
  font-size:13px;  
}
```

Server.js :

```
const express = require("express");  
const cors = require("cors");  
const db = require("./db");  
const app = express();  
app.use(cors());
```

```
app.use(express.json());  
// SAVE DATA  
app.post("/save", (req, res) => {  
  db.save(req.body, () => res.send("Saved to Cloud"));  
});  
// GET DATA  
app.get("/data", (req, res) => {  
  db.get(('_', data) => res.json(data));  
});  
app.listen(3000, () => console.log("Cloud App running"));
```

DB.js :

```
let storage = [];  
exports.save = (data, cb) => {  
  storage.push({  
    id: storage.length + 1,  
    name: data.name,  
    data: data.data  
  });  
  cb(null);  
};  
exports.get = (cb) => {  
  cb(null, storage);  
};
```

Output :

Cloud Service Comparison

AWS ▾

Services

- Compute: EC2
- Storage: S3
- Database: RDS
- Network: VPC

Pricing Calculator

9

8

Calculate Cost

Cost: \$1.06

Cloud Service Comparison

GCP ▾

Services

- Compute: Compute Engine
- Storage: Cloud Storage
- Database: Cloud SQL
- Network: VPC

Pricing Calculator

9

8

Calculate Cost

Cost: \$1.23

Cloud Service Comparison

Azure ▾

Services

- Compute: Virtual Machines
- Storage: Blob Storage
- Database: Azure SQL Database
- Network: Virtual Network

Pricing Calculator

9

8

Calculate Cost

Cost: \$1.19

RESULTS: A cloud comparison application was successfully developed to display services and simulate pay-as-you-use pricing across AWS, GCP, and Azure.

TASK23: AI PROMPT APP

AIM: To develop an application that demonstrates prompt engineering by generating and evaluating code or cloud configurations based on user input.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI with input field for user prompt
4. Capture user prompt using JavaScript
5. Send prompt to server using fetch API
6. Receive prompt data in backend (server.js)
7. Process prompt using predefined logic (db.js)
8. Generate output (code / cloud config / response)
9. Evaluate output quality and assign score
10. Send generated output and score to frontend

Program :

Index.html :

```
<!DOCTYPE html>
<html>
<head>
  <title>AI Prompt App</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="box">
  <h3>AI Prompt App</h3>
  <input id="prompt" placeholder="Enter your prompt">
  <button onclick="send()">Ask AI</button>
  <div id="out"></div>
</div>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

Style.css :

```
body {
```

```
    background:#1e1e2f;
```

```
    color:white;
```

```
    text-align:center;
```

```
    font-family:Arial;
```

```
}
```

```
.box {
```

```
    margin:100px auto;
```

```
    width:280px;
```

```
    background:#2d2d44;
```

```
    padding:20px;
```

```
    border-radius:10px;
```

```
}
```

```
input {
```

```
    width:90%;
```

```
    padding:8px;
```

```
    margin:10px;
```

```
}
```

```
button {
```

```
    padding:8px 15px;
```

```
    background:#00bcd4;
```

```
    color:white;
```

```
    border:none;
```

```
}
```

```
#out {
```

```
    margin-top:15px;
```

```
font-size:14px;  
}
```

Script.js:

```
const API = "http://localhost:3000/ask";  
function send() {  
  fetch(API, {  
    method:"POST",  
    headers:{ "Content-Type":"application/json" },  
    body: JSON.stringify({ prompt: prompt.value })  
  })  
  .then(r=>r.json())  
  .then(d=>{  
    out.innerHTML = "<b>AI:</b> " + d.reply;  
  });  
}
```

DB.js:

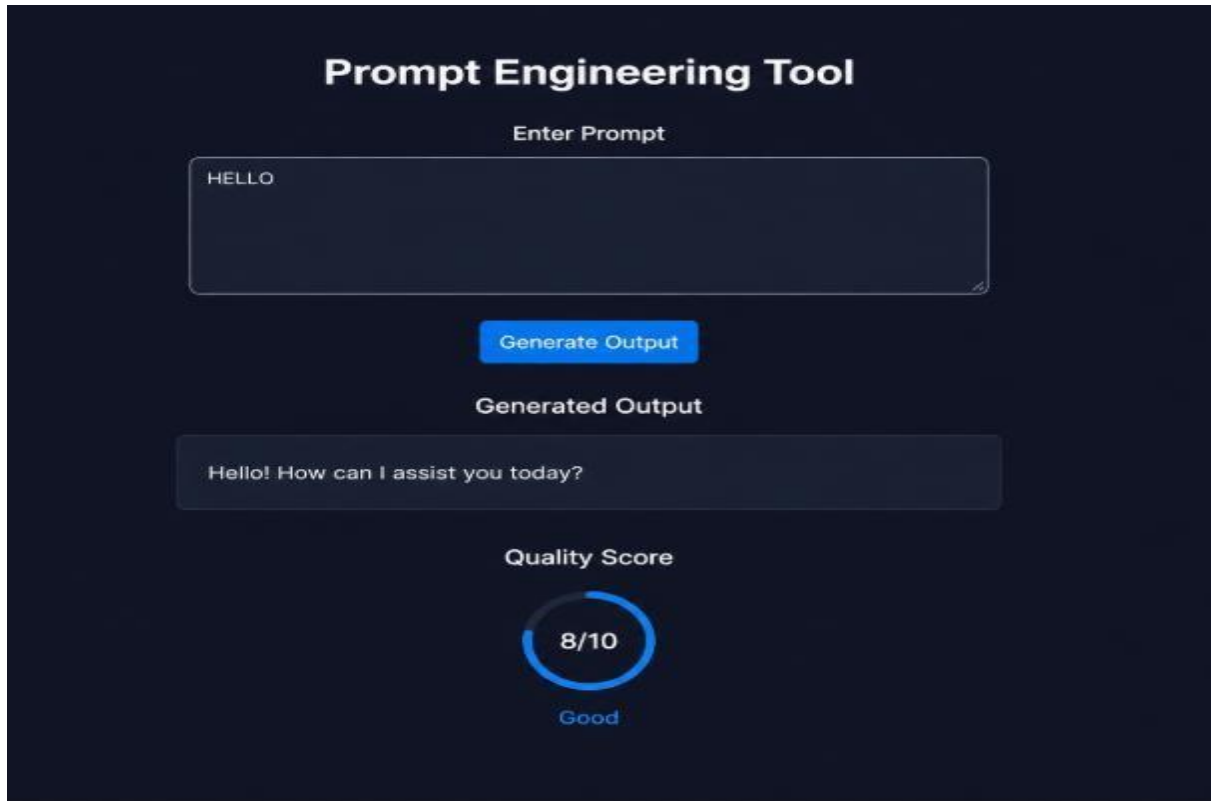
```
let history = [];  
exports.save = (data, reply, cb) => {  
  history.push({  
    id: history.length+1,  
    prompt: data.prompt,  
    reply: reply  
  });  
  cb(null);  
};  
exports.get = (cb) => cb(null, history);
```

Server.js:

```
const express = require("express");  
const cors = require("cors");
```

```
const db = require("./db");
const app = express();
app.use(cors());
app.use(express.json());
// FAKE AI RESPONSE
function generateReply(prompt) {
  return "AI Response to: " + prompt;
}
// ASK AI
app.post("/ask", (req,res)=>{
  const reply = generateReply(req.body.prompt);
  db.save(req.body, reply, ()=> {
    res.json({ reply });
  });
});
// GET HISTORY
app.get("/history", (req,res)=>{
  db.get(('_',data)=>res.json(data));
});
app.listen(3000, ()=>console.log("AI App running"));
```

Output :



The image shows a web application titled "Prompt Engineering Tool" on a dark blue background. It features a text input field with the prompt "HELLO", a blue "Generate Output" button, a text area showing the generated output "Hello! How can I assist you today?", and a "Quality Score" section with a circular progress indicator showing "8/10" and the word "Good" below it.

Prompt Engineering Tool

Enter Prompt

HELLO

Generate Output

Generated Output

Hello! How can I assist you today?

Quality Score

8/10

Good

RESULT: A prompt engineering application was successfully developed to generate and evaluate AI-based outputs for code and cloud configurations.

TASK24: Vibe Cloud App

AIM: To develop an application that uses iterative prompt engineering to generate and evaluate REST APIs, cloud deployment steps, and security configurations.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI to accept user prompts
4. Capture prompt input using JavaScript
5. Send prompt to backend using fetch API
6. Receive prompt in server and forward to processing module
7. Store prompt in history for version tracking
8. Analyze prompt type (REST API / Cloud / Security)
9. Generate corresponding output based on prompt
10. Evaluate output quality and assign score
11. Return output and score to frontend and display results
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>

<html>

<head>

  <title>Vibe Cloud App</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="card">

  <h2>☁ Vibe Cloud</h2>

  <input id="title" placeholder="Title">

  <input id="content" placeholder="Content">
```

```
<button onclick="save()">Save</button>
<button onclick="load()">Load</button>
<div id="out"></div>
</div>
<script src="db.js"></script>
<script src="script.js"></script>
</body>
</html>
```

Style.css :

```
body {
  background: linear-gradient(135deg, #1e1e2f, #3a3a6a);
  font-family: Arial;
  text-align: center;
  color: white;
}

.card {
  width: 280px;
  margin: 100px auto;
  padding: 20px;
  background: rgba(255,255,255,0.1);
  border-radius: 15px;
  backdrop-filter: blur(10px);
}

input {
  width: 90%;
  padding: 8px;
  margin: 8px;
  border-radius: 5px;
  border: none;
```

```
}  
  
button {  
  margin: 5px;  
  padding: 8px 15px;  
  background: #00bcd4;  
  border: none;  
  color: white;  
  border-radius: 5px;  
}
```

```
#out {  
  margin-top: 15px;  
  font-size: 13px;  
}
```

Script.js :

```
function save() {  
  const data = {  
  
    title: title.value,  
    content: content.value  
  };  
  addData(data);  
  load();  
}  
  
function load() {  
  getData((d) => {  
    out.innerHTML = "<pre>" + JSON.stringify(d, null, 2) + "</pre>";  
  });  
}
```


DB.js :

```
let cloud = [];  
  
function addData(data) {  
  cloud.push({  
    id: cloud.length + 1,  
    title: data.title,  
    content: data.content  
  });  
}  
  
function getData(cb) {  
  cb(cloud);  
}
```

Output :



RESULT: A Vibe Coding application was successfully developed to generate and refine cloud based features using iterative prompt engineering.

